SmogShield Lahore

Final Report: Deliverables 1–3, Phases 1–6

Problem Definition, Data Foundations, Modelling, Evaluation, and the Deliverable 3 Application and Explainability Layer

Muhammad Haider Mughal (3006) Muhammad Ayub Butt (3076)

Abdul Rehman (3059)

12 November 2026

Abstract

SmogShield is an explainable next-day air-quality forecasting system for Lahore. This report integrates the three deliverables and the six project phases: the problem definition, the data foundations, the frozen modelling core, the evaluation and bias analysis, the Deliverable 3 application, and the final evaluation. The data cover 2,113 daily rows from 23 May 2019 to 4 March 2025, built from one OpenAQ $PM_{2.5}$ monitor, daily weather, and calendar features. The modelling core is a histogram gradient-boosting regressor for next-day $PM_{2.5}$ and a six-class histogram gradient-boosting classifier for next-day AQI category, selected on a chronological validation split with a seven-day purge. On the held-out test split, the regression essentially ties persistence (RMSE 64.92 versus 64.82 $\mu g/m^3$; skill -0.0016) and the classifier is weaker than persistence on macro-F1 (0.478 versus 0.516). These results are reported without retuning on test. Deliverable 3 adds a Streamlit application with six user inputs, two category views, an 80% prediction interval (80.3% validation coverage, 77.0% test coverage), local sensitivity and global permutation importance, historical seasonal and hourly context, and a persistence comparison on every forecast. The application has 14 passing tests and a render smoke test. The project's contribution is an honest, reproducible, explainable end-to-end system, not a claim of beating a strong one-day baseline.

Question	**Answer in this report**
Does the regression beat persistence?	No. Test RMSE 64.92 versus 64.82 $\mu g/m^3$; skill -0.0016.
Does the classifier beat persistence?	No. Test macro-F1 0.478 versus 0.516.
What is new in Deliverable 3?	The application, inference, 80% intervals, explanations, seasonal context, hourly profile, ablation, tests, and report.
What does the app forecast?	Tomorrow's $PM_{2.5}$ and AQI category from six values for today.
What is the honest verdict?	A reproducible, explainable demonstrator; not an operational warning system.

Table 1: Key results at a glance.

Contents

1 Phase 1: Problem Definition **4**
1.1 Problem Statement . . . 4
1.2 Target Audience . . . 4
1.3 AI Justification . . . 4
1.4 Research Questions and Scope . . . 5

2 Phase 2: Data Foundations **6**
2.1 Data Collection . . . 6
2.2 Data Cleaning Implementation . . . 7
2.2.1 OpenAQ hourly $PM_{2.5}$ . . . 7
2.2.2 Station file . . . 7
2.2.3 Weather and modelled datasets . . . 8
2.3 Feature Strategy . . . 8
2.4 Exploratory Data Analysis . . . 9
2.5 Data Limitations and Ethical Considerations . . . 14

3 Phase 3: Models, baselines, and selection protocol **14**
3.1 Classification . . . 14
3.2 Regression . . . 15

4 Phase 3: Classification results **15**

5 Phase 3: Regression results **18**

6 Phase 4: Bias and error analysis **21**

7 Phase 4: Model limitations and recommendations **25**

8 Phase 4: Reproducibility **25**
8.1 Git-author contributions . . . 26

9 Phase 4: Evaluation summary **26**

10 Phase 5: Application and Explainability **27**
10.1 Design goals and scope . . . 27
10.2 System architecture and artifact policy . . . 27
10.3 From six inputs to 46 features . . . 29
10.4 The forecast screen . . . 30
10.5 Prediction intervals . . . 32
10.6 Explanations: global and local . . . 33
10.7 Seasonal defaults and historical context . . . 34
10.8 The hourly profile . . . 36
10.9 Persistence, guidance, and the honesty contract . . . 36
10.10 Tests and reproducibility . . . 37

11 Phase 6: Final Evaluation and Discussion **38**
11.1 Integrated results across phases . . . 38
11.2 Ablation: do weather and calendar features help? . . . 38
11.3 Backtest: 1 November 2024 . . . 39
11.4 What the app can and cannot claim . . . 41
11.5 Limitations and ethics . . . 41
11.6 Reproducibility and repository guide . . . 41
11.7 Conclusion and future work . . . 43

A Viva questions and answers **43**

B Glossary **44**

C File map and build notes **45**

1 Phase 1: Problem Definition

1.1 Problem Statement

Lahore experiences recurring episodes of severe air pollution, most visibly during the winter "smog season" that roughly spans October to February [1]. The pollutant most often used to describe these episodes is fine particulate matter ($PM_{2.5}$), particles smaller than 2.5 μm that can be inhaled deeply [2, 3].

Our own data shows how persistent the problem is. The US Diplomatic Post monitor in Lahore recorded at least 18 valid hourly readings on 1,506 days between May 2019 and March 2025. Converting each day's mean $PM_{2.5}$ to the US EPA Air Quality Index (AQI, 2012 breakpoints) [4], only 9 of those days (0.6%) fell in the "Good" category, while 1,102 days (73.2%) were "Unhealthy" or worse. These figures come from a single monitor and from days with enough valid readings, so they describe the data we have rather than the whole city.

The pain point. A resident deciding whether to cycle to work, hold a school sports session, or work outdoors tomorrow needs three things: a forecast, an idea of how reliable it is, and a reason for it. Air-quality information is commonly presented as a current reading, which leaves three gaps:

- **Timing:** a current value does not say what tomorrow will look like.
- **Trust:** a single number gives no sense of how uncertain it is.
- **Understanding:** people are rarely told which conditions (for example calm air, cold temperatures, high humidity) are driving the pollution.

Our solution. SmogShield Lahore is a web application that forecasts the next day's $PM_{2.5}$ level and AQI category for Lahore from historical air-quality and weather data (with calendar-based proxies where traffic data is unavailable). Each forecast comes with a prediction interval, the main factors the model relied on, and cautious, non-medical guidance. The project also studies recurring seasonal pollution patterns and tests whether adding weather and calendar information improves forecast accuracy.

1.2 Target Audience

The primary audience is the general public in Lahore. SmogShield is not a medical tool and does not replace official public-health advisories.

1.3 AI Justification

A rule-based system would encode expert thresholds such as "if wind speed is below X and humidity is above Y, predict poor air quality." Machine-learning models have already been used to forecast Lahore's AQI from historical series [5]; SmogShield extends this line by forecasting next-day $PM_{2.5}$ with weather and calendar inputs and by attaching uncertainty and explanations to each forecast. We argue that machine learning is better suited to this problem for the following reasons.

1. **Many interacting factors.** Tomorrow's $PM_{2.5}$ depends on today's level and on wind speed and direction, humidity, dew point, temperature, pressure, rainfall and the time of year. The effect of one factor depends on the others (for example, calm air matters most when it is also cold and humid). Writing and justifying thresholds for every combination by hand does not scale, whereas learning algorithms such as tree ensembles fit these interactions directly from data.

User group	Decision they face	What SmogShield provides
Commuters, cyclists and runners	Whether to travel or exercise outdoors tomorrow	Next-day AQI category with an uncertainty range
Parents and schools	Whether to hold outdoor activities	Category-based, cautious guidance
Outdoor workers	Planning the working day	Forecast and typical cleaner hours from historical patterns
Residents who may be sensitive to air pollution, and their carers	Deciding how cautious to be	The forecast category and the factors behind it (not medical advice)
Students and researchers	Assessing a transparent baseline	Documented data, models and limitations

Table 2: Target users and how the application supports their decisions.

2. **Relationships change with the season.** The same wind speed does not mean the same thing in a monsoon July and in a December smog episode. Rules would need separate hand-tuned sets for each regime. A model trained on several years of data can learn these differences.

3. **Uncertainty has to be estimated.** A fixed rule gives a single answer. Learned models allow prediction intervals to be estimated from historical forecast errors (for example with quantile regression), which supports the trust gap described above.

4. **The data is suitable.** We have an observed daily $PM_{2.5}$ series with 1,506 usable days, together with more than 2,600 days of daily weather. This is a tabular supervised learning problem with enough rows for tree-based models and regularised linear models, and possibly a small recurrent network for comparison.

5. **Maintainability.** When new data arrives, a model can be retrained and re-evaluated. A rule set must be re-tuned manually.

6. **Complement to physical models.** Atmospheric models exist, but a modeled air-quality dataset we examined during Phase 2 read about half as high as the monitor on overlapping days and under-read smog peaks. A statistical model trained on local observations is a practical complement.

An honest counterpoint. Daily pollution is strongly persistent, so a simple baseline that predicts "tomorrow equals today" is hard to beat. For this reason we compare every learned model against that persistence baseline and against simple linear models. We claim a benefit from machine learning only where the evaluation shows one.

1.4 Research Questions and Scope

RQ1 Can machine learning models forecast next-day $PM_{2.5}$ and AQI category better than a persistence baseline?

RQ2 Does adding weather, and then calendar information, improve accuracy compared with air-quality history alone (ablation study)?

RQ3 Where do the models fail? We examine errors by season and on extreme-pollution days.

Aspect	Rule-based	Machine learning
Factor interactions	Hand-written, limited	Learned from data
Seasonal differences	Separate rule sets	Learned, can use season as input
Uncertainty	Not provided	Estimated from past errors
Updating	Manual re-tuning	Retrain on new data
Evidence of accuracy	Hard to measure fairly	Compared with baselines on held-out data

Table 3: Why a learned model is preferred over fixed rules for this problem.

Scope and limitations. The forecast is for the next day at daily resolution, for Lahore, based mainly on one reference monitor. Traffic is represented by calendar proxies because no open, time-aligned Lahore traffic dataset was found. The data has gaps, and the evaluation uses a time-based split so that no future information leaks into training. The application gives cautious guidance and is not medical advice.

2 Phase 2: Data Foundations

Phase 2 turns the problem definition into a documented, reproducible dataset. This section records where every file came from, what was cleaned and why, how variables are encoded for modelling, and what the exploratory analysis shows about Lahore's $PM_{2.5}$.

2.1 Data Collection

Sources. Table 4 lists the five data products. We combine a measured target series with meteorological features and two modelled products that serve as validation; the modelled products are never treated as ground truth.

Dataset	Source and access	Coverage	Rows	Licence
OpenAQ hourly $PM_{2.5}$	OpenAQ v3 API, sensor 25135 (US Diplomatic Post, Lahore)	22 May 2019 – 4 Mar 2025	46,465 hourly	Provider-specific
ERA5 daily weather	Open-Meteo Historical Weather API, model `era5`	1 Jun 2019 – 7 Oct 2026	2,686 daily	CC BY 4.0
Station AQI + weather	GitHub `taimoorbarlas15/Lahore_Air_Quality`	1 Jun 2019 – 30 Nov 2023	1,644 daily	Not stated
CAMS hourly (modelled)	Kaggle `aadbutt/dataset`; regenerable from the Open-Meteo Air-Quality API	1 Jan 2023 – 1 May 2025	20,448 hourly	Unknown / CC BY 4.0
CAMS daily (modelled)	Zenodo DOI 10.5281/zenodo.18673725	1 Aug 2022 – 18 Feb 2026	1,298 daily	CC0-1.0

Table 4: Data sources used in Phase 2.

How the data was obtained. The API-based sources are downloaded by `data_scripts/01_download_data.py`, which uses only the Python standard library. OpenAQ data is requested in 90-day chunks with pagination and retries; the key is read from the `OPENAQ_API_KEY` environment variable and is never written into the code or the repository. The Open-Meteo requests are chunked where a

single response would be too large. The Zenodo archive and the GitHub station file are downloaded directly.

Reproducibility and storage. Every delivered raw file is fingerprinted with SHA-256 in `reports/data_collection_log.md`. Re-running the collection script reproduced the station file and all three Zenodo files byte-for-byte; the ERA5 and CAMS sources are live and may be revised slightly, so they are equivalent rather than necessarily identical. Raw files are kept on the shared Google Drive folder and are excluded from GitHub; only scripts, documentation and derived outputs are version-controlled.

Why several sources. OpenAQ provides the measured target. ERA5 supplies the weather features that Phase 1 identified as physically relevant. The GitHub station file offers an independent cross-check of AQI and weather over 2019–2023: daily AQI computed from the OpenAQ series agrees with it over 1,153 shared days, with a correlation of 0.99 and a mean absolute difference of 9.3 AQI points. The two CAMS products quantify how far a modelled air-quality product is from the monitor, which is useful context for the "complement to physical models" argument in Phase 1.

2.2 Data Cleaning Implementation

Scope. Cleaning covers missing values, duplicate records, impossible or extreme values, and the standardisation of dates and units. Combining columns is feature engineering and is described in Section 2.3. Missing values are left as missing rather than imputed: fabricating a smog peak would bias both the forecast and the evaluation.

Code. The steps below are implemented in `data_scripts/`: `02_clean_pm25_simple.py` and `03_pm25_aqi_and_eda.py` (OpenAQ series, daily aggregation, AQI and the EDA charts), `04_clean_station_simple.py` (station file), three scripts numbered 05–07 (ERA5 weather and the two CAMS files; the CAMS hourly timestamps are UTC and are converted to Pakistan time), `08_build_calendar.py` (calendar table) and `09_build_features.py` (feature table). Each script prints before-and-after counts.

2.2.1 OpenAQ hourly $PM_{2.5}$

The raw file contains 46,465 hourly records. There are no duplicate timestamps, no negative values and no values above 1,000 $\mu g/m^3$; the maximum is 974 $\mu g/m^3$, which is a plausible extreme smog reading rather than a data error. A total of 8,997 values (19.4%) are empty, a further 4,238 hours between the first and last timestamp have no record at all, and 9,596 records carry OpenAQ quality flags, of which 1,018 still contain a value.

We aggregate the hourly series to local calendar days (Pakistan Standard Time, UTC+5). A day is *usable* when it has at least 18 valid hourly readings; this leaves 1,506 usable days out of the 2,113 calendar days spanned by the file. The daily value is the mean of the valid hours, and the daily AQI is computed from that mean with the US EPA 2012 breakpoints. Flagged values are retained for now; a sensitivity test that excludes them is planned for the modelling phase.

2.2.2 Station file

The station file has 1,644 daily rows and 17 columns. Three problems were fixed. First, dates are stored as `dd-mm-yy`; they were parsed with an explicit `%d-%m-%y` format, because allowing pandas to infer the format misparsed 594 of the 1,644 days. Second, temperatures are in Fahrenheit, wind speeds in miles per hour and pressure in inches of mercury; these were converted to degrees Celsius,

kilometres per hour and hectopascals. Third, zeros in the minimum-temperature, minimum-dew-point, minimum-humidity and minimum-pressure columns are physically impossible in Lahore (53, 53, 61 and 127 cells respectively), and spikes in the maximum columns (for example a maximum temperature of 196 °F) fall outside physical limits; in total 323 impossible values were set to missing. Gaps of at most three days in the daily average weather columns were filled from the previous day. The AQI itself is never filled: only 2 of the 221 missing AQI values fall on days with a usable $PM_{2.5}$ value in the OpenAQ series, so they stay missing.

2.2.3 Weather and modelled datasets

The ERA5 file contains three metadata lines followed by 2,686 daily rows; it has no duplicate dates and no missing cells after the empty trailing row for 8 October 2026 is dropped. The CAMS hourly file has 20,448 rows with no duplicates, missing cells or negative values. The CAMS daily file has 1,298 rows and 35 missing cells; these are left as missing because the file is used only for comparison.

Dataset	Issue detected	Action taken
OpenAQ hourly	8,997 missing values (19.4%)	Left missing; day needs ≥18 valid hours
OpenAQ hourly	9,596 flagged records	Retained; sensitivity test planned
OpenAQ hourly	No duplicates or impossible values	None required
Station daily	Dates as `dd-mm-yy`	Parsed with explicit day-first format
Station daily	Fahrenheit, mph, inches	Converted to °C, km/h, hPa
Station daily	221 missing AQI values	Left missing (only 2 recoverable)
Station daily	323 impossible values (zeros in minima, spikes in maxima)	Set to missing
ERA5 daily	One empty trailing row	Dropped
CAMS hourly	No issues found	None required
CAMS daily	35 missing cells	Left missing (comparison only)

Table 5: Cleaning actions and their effect.

After cleaning, the measured target series has 2,113 daily rows, of which 1,506 are usable for modelling and 607 (28.7%) are missing.

2.3 Feature Strategy

Target. The forecasting target is the next day's daily mean $PM_{2.5}$, defined as `target_next_pm25 = pm25.shift(-1)`. Because the daily distribution is strongly right-skewed (median 88, 95th percentile 331 and maximum 630 $\mu g/m^3$), the models are also given a log-transformed target, `target_next_log` $= \log(1 + PM_{2.5})$. Predictions can be transformed back with the inverse. For the classification task, `target_next_aqi_category` is the next day's AQI category.

Feature groups. Table 6 summarises the 63-column feature table written to `data/processed/features_daily.csv` by `09_build_features.py`. Every historical feature is shifted so that it would have been available at forecast time. After requiring a next-day target and lags 1–3, 1,179 rows remain (1,173 when weather is also required); this is the number of days available to the models.

Normalisation and scaling. Meteorological variables have different units and ranges, and $PM_{2.5}$ contains extreme smog peaks. We therefore use a robust scaler, subtracting the median and dividing

Group	Variables and purpose
Air-quality history	Today's value, $\log(1 + \mathrm{PM}_{2.5})$, lags 1–3, rolling means over 3 and 7 days, and the one-day difference. These capture the strong day-to-day persistence of pollution.
Weather	Mean, maximum and minimum temperature; mean humidity; dew point; surface pressure; mean and maximum wind speed; dominant wind direction; precipitation; shortwave radiation; cloud cover.
Weather lags	Temperature, humidity, wind speed and pressure from the previous day, because yesterday's weather is known when tomorrow's forecast is issued.
Calendar	Year, month, day of year, weekday; weekend, Ramadan, Eid and public-holiday flags (a simplified list; the fuller table from `08_build_calendar.py` is kept separately). These are proxies for traffic and behaviour, since no open, time-aligned Lahore traffic dataset was found.
Cyclical encoding	Sine and cosine of month, day of year and weekday. This preserves the fact that December is adjacent to January and that Sunday is adjacent to Monday.
Season encoding	One-hot indicators for winter, spring, summer and autumn (meteorological seasons).
Scaled weather	Robustly scaled copies of the meteorological variables (see below).

Table 6: Feature groups in the daily modelling table.

by the interquartile range, rather than the mean and standard deviation. The resulting scaled columns have median zero and unit interquartile range, and a single extreme day cannot dominate them. In the feature table the median and interquartile range are computed on all rows; they are re-fitted on the training years only when models are trained, so that statistics from the test period do not leak into training. $\mathrm{PM}_{2.5}$ itself is handled with the log transform described above rather than with standardisation.

Encoding. Calendar variables are encoded twice. The raw values (month 1–12, weekday 0–6) are kept for interpretability, and sine/cosine pairs are added so that a linear model sees a smooth cycle. Season is one-hot encoded because it is nominal, not ordered. Weekend, Ramadan, Eid and public-holiday flags are already binary.

Missing values and leakage. Missing $\mathrm{PM}_{2.5}$ values are not imputed. Tree-based models can handle missing features natively; for linear models the plan is median imputation with an explicit missing-value indicator. All lags and rolling means are computed after shifting, so no value from the target day enters the features. The evaluation will use a time-based split (training on earlier years and testing on later ones), never a random split, so that the model cannot learn from the future.

2.4 Exploratory Data Analysis

Distribution and seasonality. Figure 1 shows the daily $\mathrm{PM}_{2.5}$ distribution: it is unimodal but heavily right-skewed, with a long tail of severe episodes. Figure 2 shows the seasonal cycle. Winter (December–February) averages 219 $\mu\mathrm{g/m}^3$, autumn 146, spring 78 and summer 55. The winter peak

coincides with the smog season described in Phase 1 and with the cold, calm, high-pressure conditions that trap pollution near the ground.

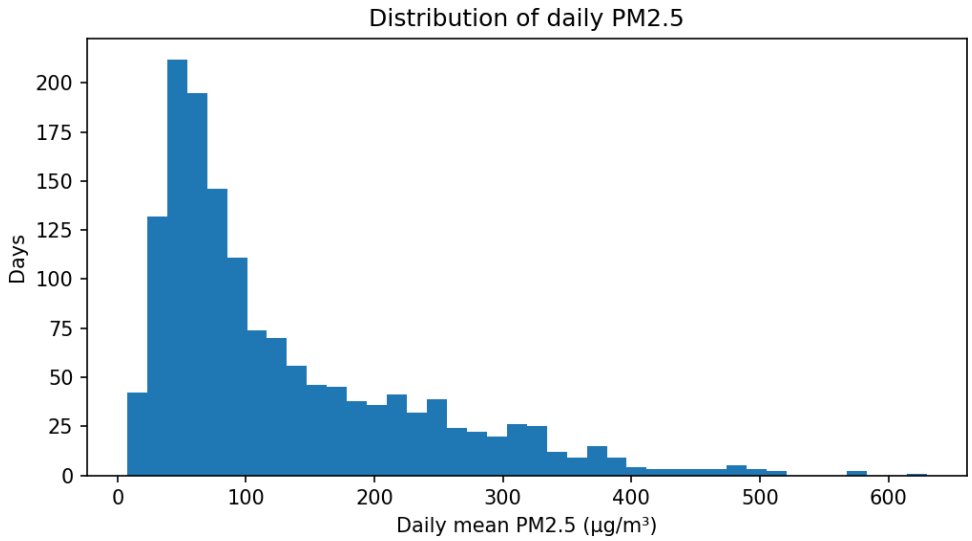


Figure 1: Distribution of daily mean $PM_{2.5}$ for usable days, Lahore 2019–2025.

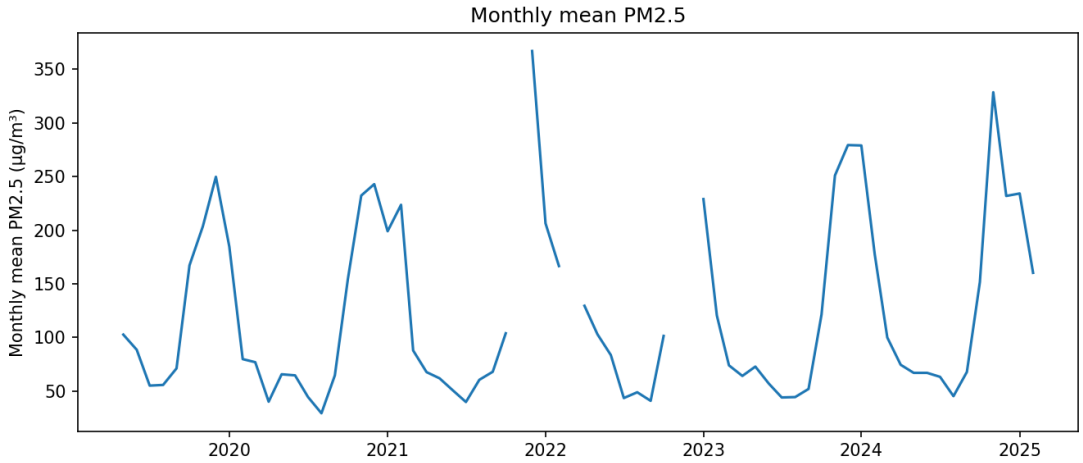


Figure 2: Monthly mean $PM_{2.5}$. The winter peaks dominate the series; gaps are months with too few usable days.

Data coverage. Coverage is uneven. Figure 3 shows the percentage of usable days in each month: some months are complete, while parts of 2022 and 2023 have very few usable days. This matters for evaluation, because a model trained mostly on winter smog will see a different mix of conditions in a sparse summer month. The time-based split and the per-season error analysis in Phase 3 are designed with this in mind.

AQI categories. Figure 4 and Table 7 show the AQI class distribution. "Unhealthy" is the most common class (647 days, 43.0%), and 1,102 days (73.2%) are "Unhealthy" or worse. Only 9 days (0.6%)

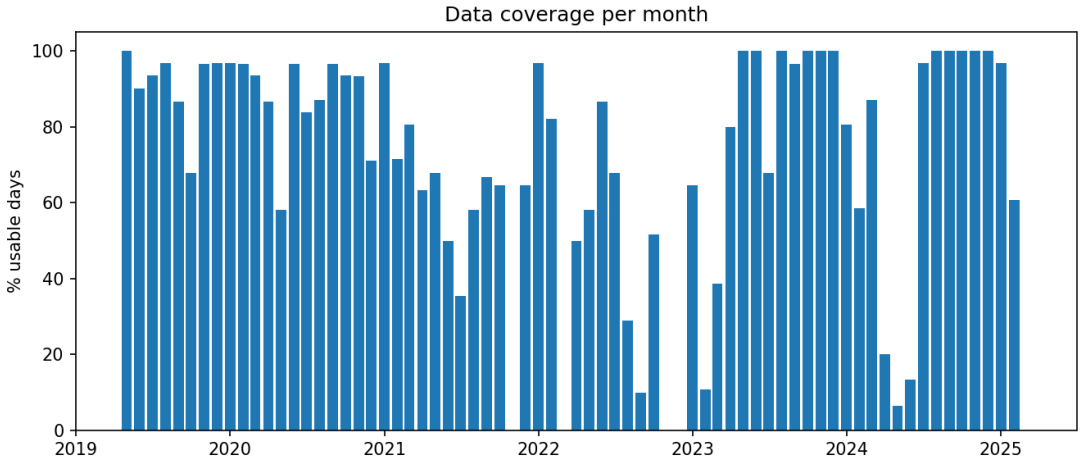


Figure 3: Percentage of usable days per month. Gaps are months with insufficient valid hourly readings.

are "Good". The classes are therefore strongly imbalanced, so overall accuracy would be misleading; Phase 3 will report macro-averaged F1 and per-class recall alongside accuracy.

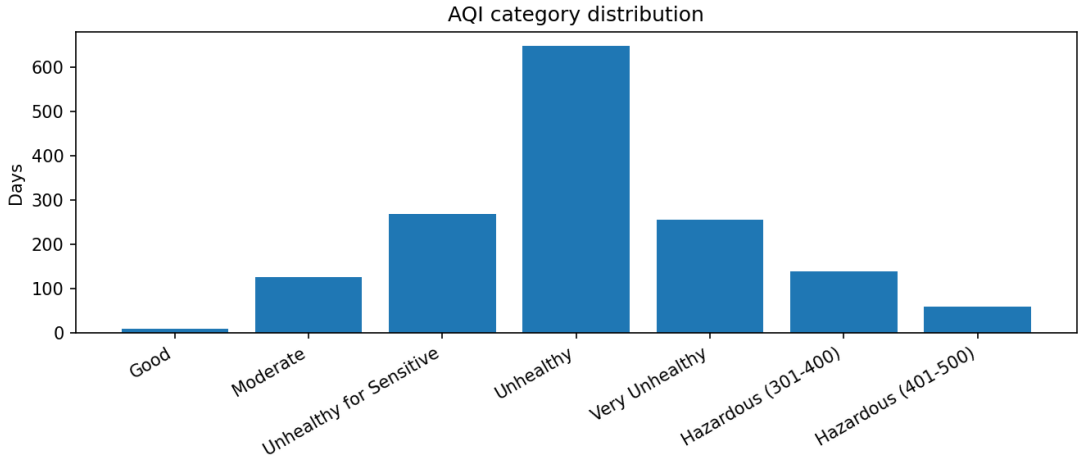


Figure 4: Daily AQI category counts for usable days.

Relationships with weather. Figure 5 reports Pearson correlations between daily $PM_{2.5}$ and the meteorological variables. The strongest associations are negative with temperature ($r = -0.64$) and shortwave radiation ($r = -0.50$), and positive with surface pressure ($r = +0.64$). Wind speed is negatively associated ($r = -0.38$) and humidity positively ($r = +0.30$). Figure 6 shows the same relationships as scatter plots with linear fits. These patterns are physically consistent: cold, calm, high-pressure days with weak sunlight and little ventilation allow pollution to accumulate, while wind and rain disperse it.

What the EDA implies for modelling. Three points follow. First, the seasonal cycle is strong enough that season and calendar features are necessary, not optional. Second, persistence is likely to be the hardest baseline to beat, which is why Phase 1 commits to reporting it. Third, the weather

AQI category	Days	Share
Good	9	0.6%
Moderate	126	8.4%
Unhealthy for Sensitive Groups	269	17.9%
Unhealthy	647	43.0%
Very Unhealthy	256	17.0%
Hazardous (301–400)	139	9.2%
Hazardous (401–500)	60	4.0%
Total	1,506	100%

Table 7: AQI category distribution for the 1,506 usable days.

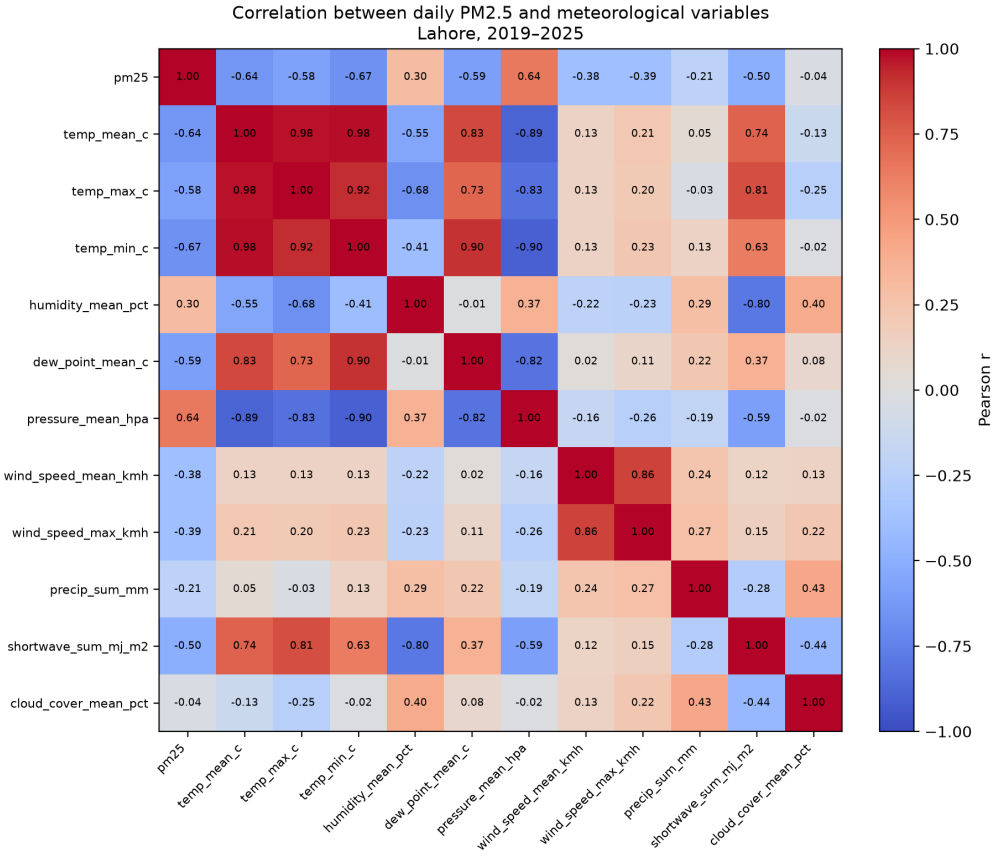


Figure 5: Pearson correlation matrix for daily $PM_{2.5}$ and meteorological variables.

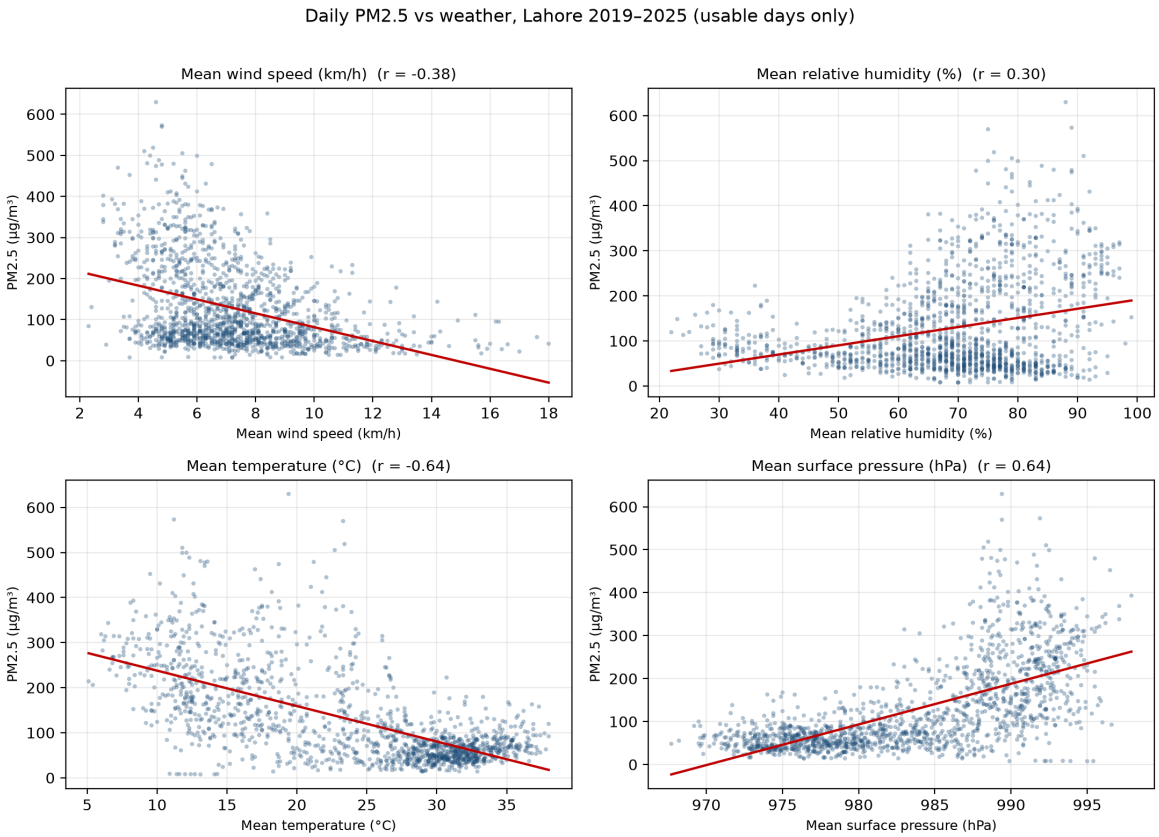


Figure 6: Daily $PM_{2.5}$ against wind speed, humidity, temperature and pressure, with linear fits.

relationships are correlations, not causal effects; the models will be evaluated on held-out years rather than on their ability to fit these patterns.

2.5 Data Limitations and Ethical Considerations

- **One monitor, not a city.** The target comes from a single US Diplomatic Post monitor. It is a useful reference series but does not represent every neighbourhood in Lahore.
- **Substantial gaps.** 607 of 2,113 calendar days (28.7%) do not have enough valid hours. Coverage is uneven across months, so evaluation must report results by season as well as overall.
- **Station file gaps.** The station AQI is missing on 221 of 1,644 days (13.4%), including a 47-day gap in the 2021 smog season.
- **Modelled data is not measured data.** The CAMS products are used for comparison only. On the 632 days shared with the monitor, the modelled daily series averaged 0.54 of the monitor's level (median daily ratio 0.61) and under-read smog peaks, so it cannot substitute for observations.
- **Calendar proxies are approximate.** Ramadan and Eid dates follow published calendars but can shift by a day with moon sighting; public holidays are a simplification. Traffic is not observed directly.
- **Licences.** The Zenodo dataset is CC0-1.0 and Open-Meteo/ERA5 is CC BY 4.0. The Kaggle `aadbutt` dataset states an unknown licence, and the GitHub station repository states no licence. We therefore attribute both sources and keep raw files out of the public repository rather than redistributing them.
- **Not medical advice.** The application provides cautious, category-based guidance and does not replace official health advisories.

3 Phase 3: Models, baselines, and selection protocol

3.1 Classification

The classification candidates were:

1. Majority class.
2. Persistence: tomorrow's category equals today's category.
3. Logistic regression with class weights.
4. Histogram gradient boosting (HGB).
5. An optional small MLP, retained only if it improved validation performance.

The persistence baseline uses the majority training class code, 2 (Unhealthy), as its fallback when today's category is missing. This fallback must be stated whenever the persistence result is reported. The MLP was excluded after validation macro F1 of 0.4124 did not exceed HGB's 0.4941.

Hyperparameters were tuned with five-fold `TimeSeriesSplit` on the raw training frame, with a seven-day gap. Tuning never cross-validated on the already-preprocessed training matrix. The final model was refitted on the training split only, not on train plus validation. The test set was then

touched once for the final model. The classification selection metric was validation macro F1 over all six configured labels. The CV score is computed per fold over the classes present in that fold, so it is not directly comparable with the six-label validation score; that caveat is recorded in the metrics file.

Table 8: Selected hyperparameters. Classification used five-fold time-series CV with a seven-day gap and selected on validation macro F1; regression selected on validation RMSE.

Task	Model	Selected parameters	Validation metric	Status
Classification	LogisticRegression	`{"C": 0.1, "class_weight": "balanced", "max_iter": 500, "penalty": "l1"}`	macro F1 = 0.4665	completed
Classification	HistGradientBoosting	`{"l2_regularization": 0.1, "learning_rate": 0.1, "max_iter": 100, "max_leaf_nodes": 31, "min_samples_leaf": 10}`	macro F1 = 0.4941	selected
Classification	MLP	`{"alpha": 0.01, "early_stopping": true, "hidden_layer_sizes": [128, 64, 32], "learning_rate": "constant", "max_iter": 500, "n_iter_no_change": 15, "validation_fraction": 0.1}`	macro F1 = 0.4124	excluded
Regression	Ridge	`{"alpha": 10.0}`	RMSE = 45.6916	not selected
Regression	HGB	`{"learning_rate": 0.05, "max_depth": 3, "max_iter": 200}`	RMSE = 45.6243	selected

3.2 Regression

The regression candidates were persistence, Ridge regression, and histogram gradient boosting. Hyperparameters were tuned with the same five-fold time-series procedure and a seven-day gap. Model selection used validation RMSE. The selected model was refitted on training rows only and evaluated once on test. Persistence is not imputed for missing current-day PM2.5; it is scored on the same scorable rows as the models. The final code calls the metric helper with `allow_missing=False` for every method. The option `allow_missing=True` remains reserved for an unfiltered persistence baseline only; the persistence `n_dropped` values are 6 validation and 1 test, the same rows on which every method is scored.

The selected regression model was HGB, with validation RMSE 45.6243; Ridge was close behind at 45.6916. The test comparison is reported in Section 5 without changing the selection rule.

4 Phase 3: Classification results

Table 9 reports validation and test metrics for the majority, persistence, logistic regression, and HGB candidates. HGB was selected on validation macro F1 (0.4941 versus 0.4665 for logistic regression). On test, HGB reached macro F1 0.4780, below persistence's 0.5157. HGB's test accuracy was higher than persistence (0.5856 versus 0.5676), but accuracy is dominated by the larger classes; macro F1 exposes the weaker performance on rare classes.

The selected HGB model's test AUC-ROC is 0.8886 and logistic regression's is 0.8719. The one-vs-rest macro AUC is computed from the aligned six-column probability matrix. Rows sum to one within the helper's tolerance. The majority baseline has no probability output, so its AUC is reported as undefined rather than filled with a fake value. HGB's test AUC (0.8886) is higher than its validation AUC (0.8765) even though its test macro F1 is below persistence. AUC measures probability ranking

Table 9: Classification results on the chronological validation and test splits. Macro F1 is the selection metric.

Model	Split	Acc.	Bal. acc.	Macro P	Macro R	Macro F1	Wtd. F1	AUC
Majority class	Validation	0.3033	0.1667	0.0505	0.1667	0.0776	0.1411	–
Majority class	Test	0.3468	0.1667	0.0578	0.1667	0.0858	0.1786	–
Persistence	Validation	0.5697	0.4760	0.4772	0.4760	0.4766	0.5691	–
Persistence	Test	0.5676	0.5157	0.5157	0.5157	0.5157	0.5676	–
Logistic regression	Validation	0.5287	0.4995	0.4775	0.4995	0.4665	0.5521	0.8842
Logistic regression	Test	0.5225	0.5062	0.4673	0.5062	0.4601	0.5337	0.8719
HGB	Validation	0.5943	0.4976	0.5049	0.4976	0.4941	0.5824	0.8765
HGB	Test	0.5856	0.4776	0.4804	0.4776	0.4780	0.5815	0.8886

rather than the thresholded label decision, and persistence has no probability output, so the two metrics answer different questions. Persistence's test macro precision, recall, and F1 are all 0.5157, and its weighted F1 equals accuracy (0.5676), because predicted and true class counts coincide exactly on the test rows. This is not a copy error; it is a property of the persistence shift on this series.

Table 10 gives the selected HGB model's test per-class results. Good/Moderate has support 8 and Hazardous 401–500 has support 19; both should be read as noisy. The largest class, Unhealthy, has F1 0.7013, while Very Unhealthy has F1 0.7040. The two Hazardous bands are weaker: 0.3600 and 0.2353.

Table 10: Per-class test performance of the selected HistGradientBoosting classifier. Good/Moderate and Hazardous (401–500) have small support and should be treated as noisy estimates.

Category	Support	Precision	Recall	F1
Good/Moderate	8	0.3750	0.3750	0.3750
Unhealthy for Sensitive	34	0.5161	0.4706	0.4923
Unhealthy	77	0.7013	0.7013	0.7013
Very Unhealthy	60	0.6769	0.7333	0.7040
Hazardous (301-400)	24	0.3462	0.3750	0.3600
Hazardous (401-500)	19	0.2667	0.2105	0.2353

Table 11 gives the test confusion matrix. The model frequently predicts Unhealthy when the truth is Unhealthy for Sensitive, and it confuses the two Hazardous bands with each other and with Very Unhealthy. This is consistent with ordered severity classes and with the class imbalance.

Table 11: Selected HGB test confusion matrix. Codes: 0 Good/Moderate, 1 Unhealthy for Sensitive, 2 Unhealthy, 3 Very Unhealthy, 4 Hazardous (301–400), 5 Hazardous (401–500).

True	Pred 0	Pred 1	Pred 2	Pred 3	Pred 4	Pred 5
0	3	2	3	0	0	0
1	4	16	14	0	0	0
2	1	13	54	6	2	1
3	0	0	6	44	7	3
4	0	0	0	8	9	7
5	0	0	0	7	8	4

Training curves. A per-epoch training-loss curve exists only for the MLP, which was excluded because its validation macro F1 (0.4124) did not exceed that of HGB (0.4941). For logistic regression (full-batch solver) and HGB (no per-epoch loss recorded in our configuration) we report validation-

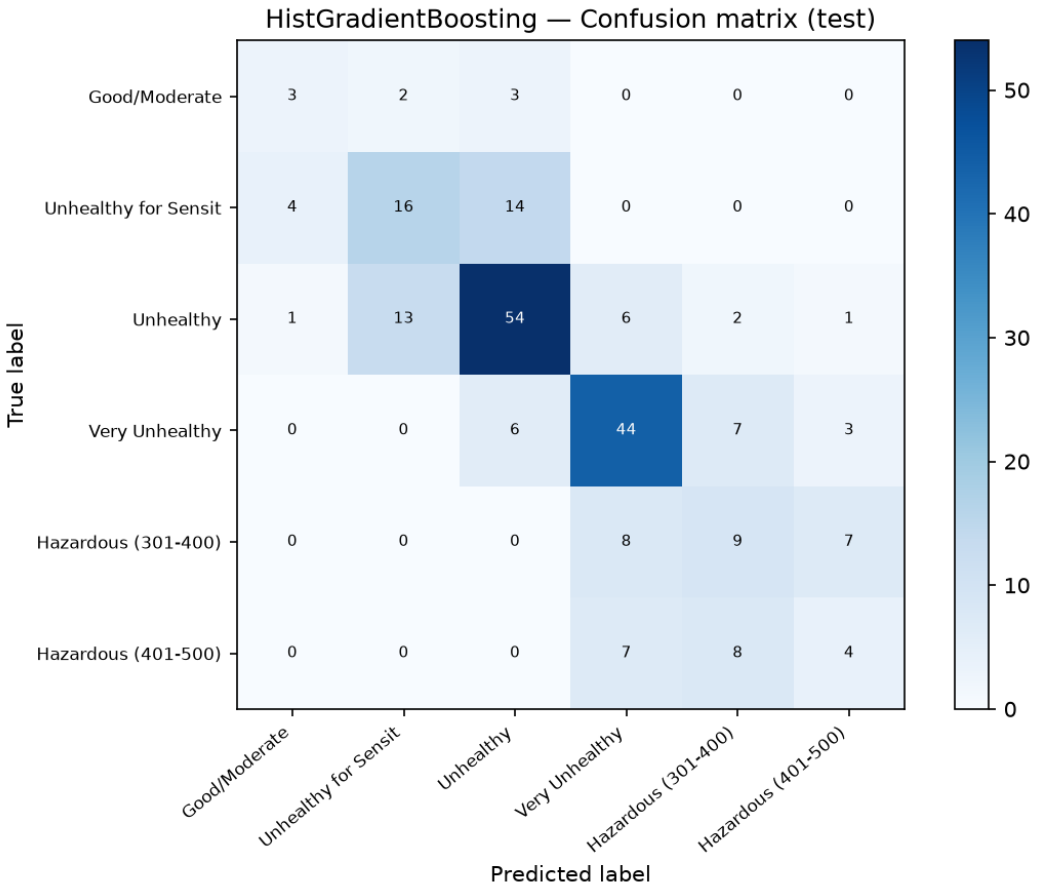


Figure 7: Selected HGB test confusion matrix. The raw matrix is also given in Table 11.

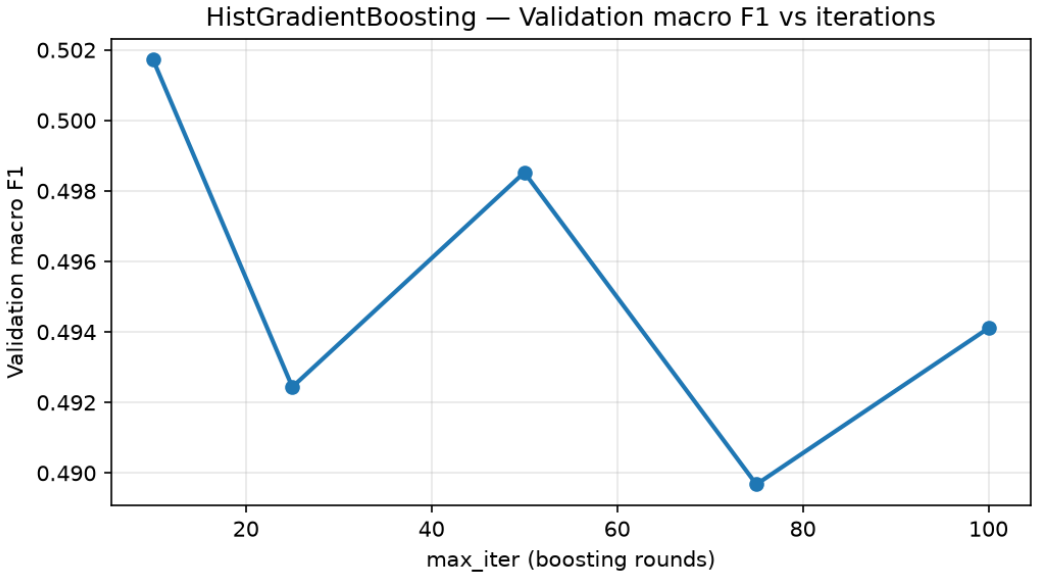


Figure 8: Validation macro F1 for the HGB configuration used in model selection.

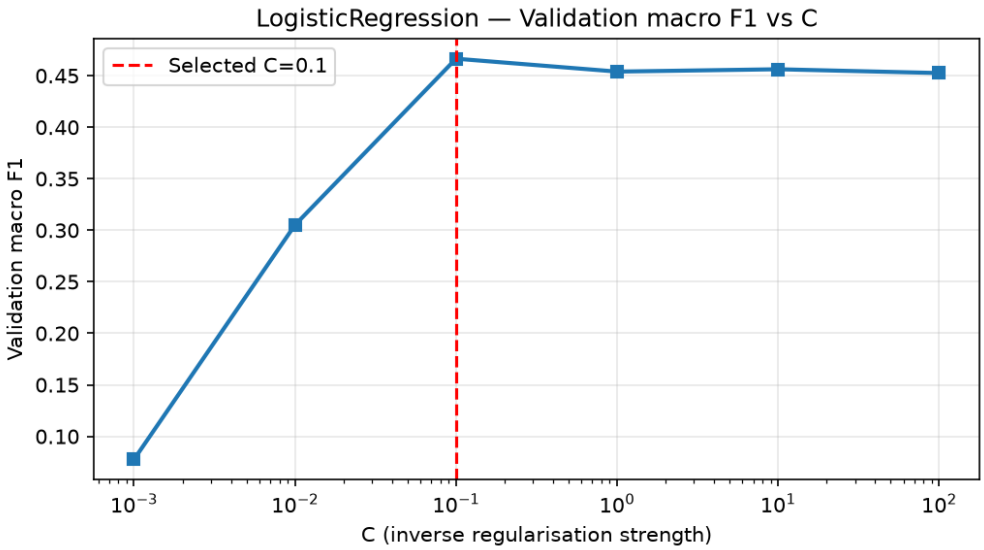


Figure 9: Logistic-regression validation curve over the regularisation strength C. The selected value was $C = 0.1$.

score curves instead (Figures 8, 9, and 14). Figure 8 is nearly flat (macro F1 between about 0.490 and 0.502), so the iteration count is not a sensitive parameter.

5 Phase 3: Regression results

Table 12 reports the regression comparison. On validation, HGB had the lowest RMSE (45.6243), narrowly ahead of Ridge (45.6916); both beat persistence (51.5457) by about 11.4–11.5%. On test, however, HGB reached 64.9198, essentially tied with persistence (64.8175, a 0.10 RMSE gap); its skill score was −0.0016. Ridge reached 62.0956 on test and beat persistence, with skill 0.0420. R2 follows the same pattern: Ridge's test R^2 is 0.7356, HGB's is 0.7110, and persistence's is 0.7119.

This is a genuine validation-to-test reversal, not a reporting error. The selection rule chose HGB on validation RMSE before test was touched. The report keeps that selection and reports the test outcome honestly. It does not switch the final model to Ridge after seeing test.

Table 12: Regression results on the chronological validation and test splits. Skill is $1 - \text{RMSE}_{model}/\text{RMSE}_{persistence}$.

Model	Split	RMSE	MAE	R^2	Skill	n
Persistence	Validation	51.5457	33.5298	0.7808	–	244
Persistence	Test	64.8175	39.9777	0.7119	–	222
Ridge	Validation	45.6916	31.8794	0.8277	0.1136	244
Ridge	Test	62.0956	38.4750	0.7356	0.0420	222
HGB	Validation	45.6243	29.9781	0.8282	0.1149	244
HGB	Test	64.9198	40.7740	0.7110	-0.0016	222

All models are scored on the same 244 validation and 222 test scorable rows; the 6 validation and 1 test rows dropped for missing current-day PM2.5 are not scored.

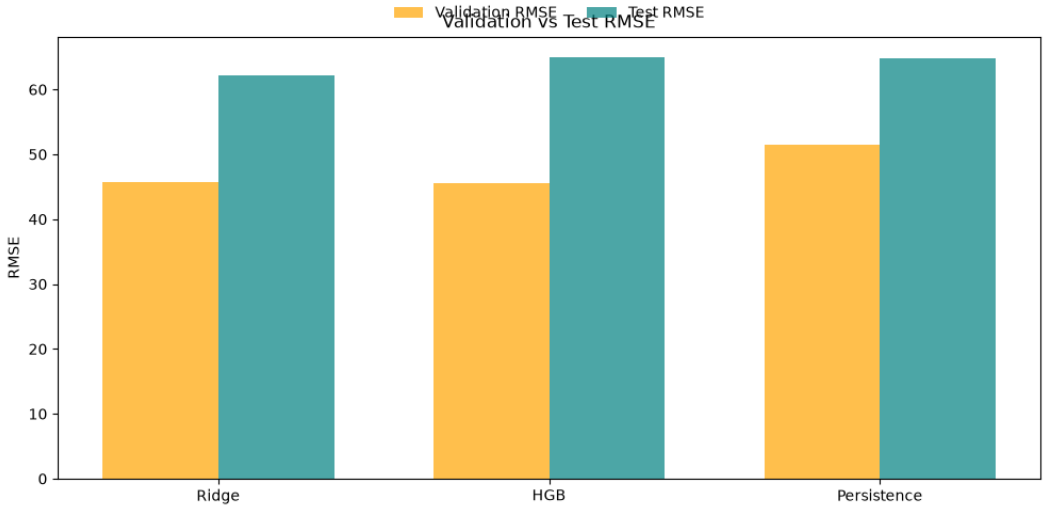


Figure 10: Regression metric comparison across candidate models and splits.

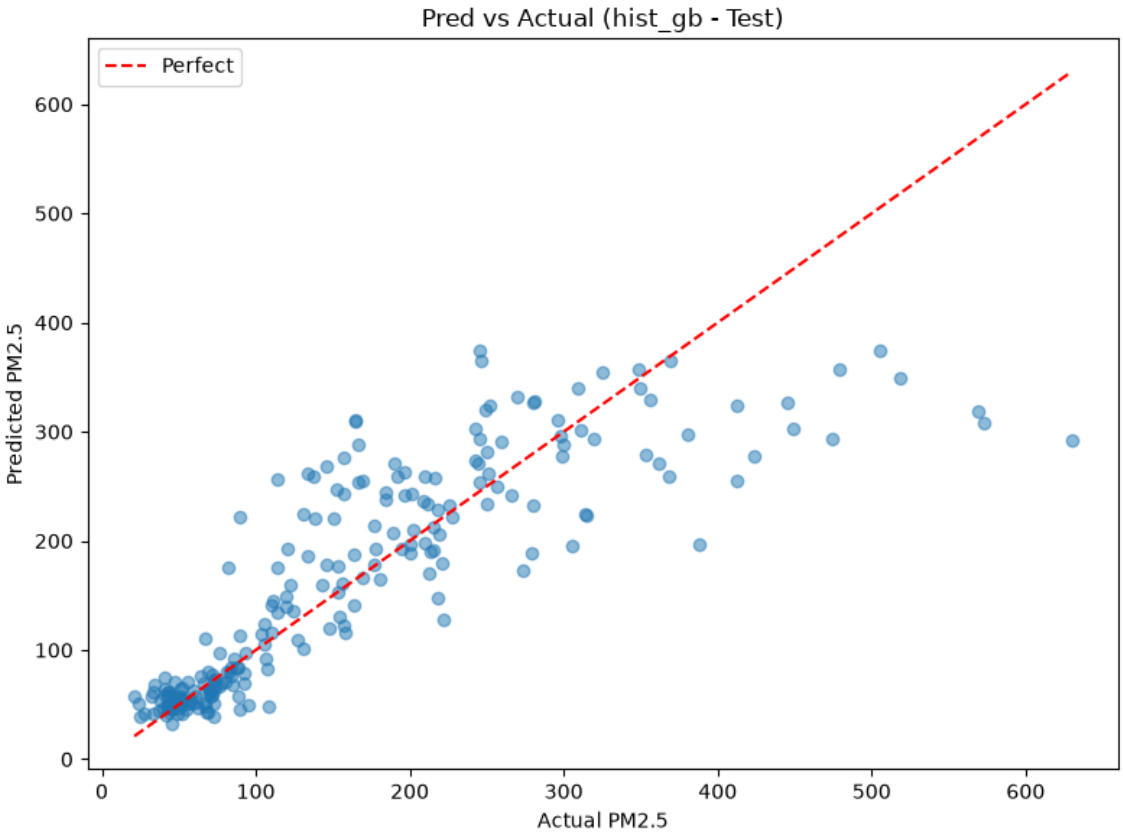


Figure 11: Selected HGB regression: predicted versus actual next-day PM2.5.

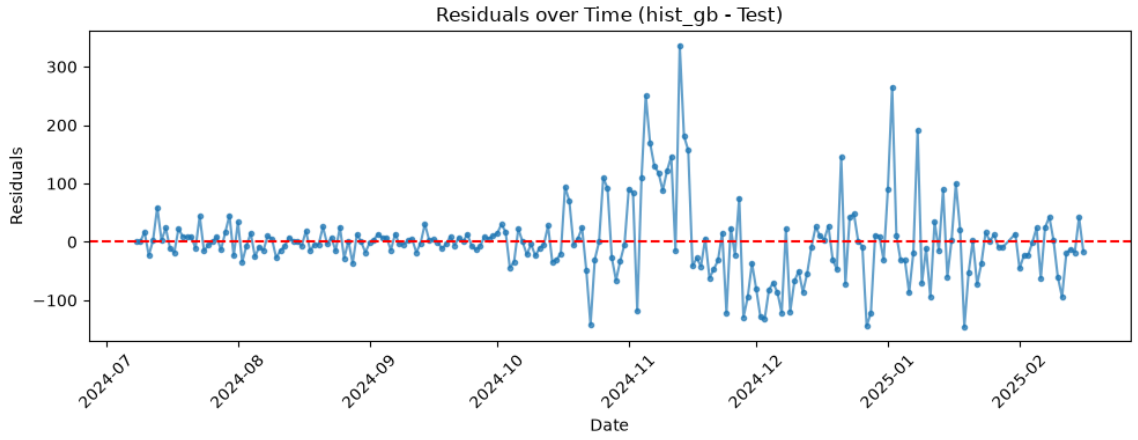


Figure 12: Selected HGB regression residuals over time.

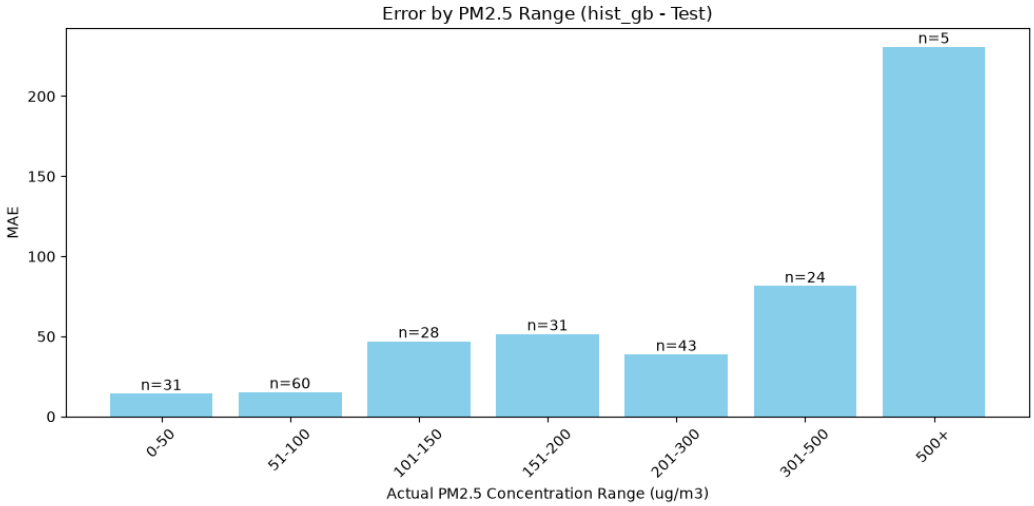


Figure 13: Selected HGB regression error by pollution severity.

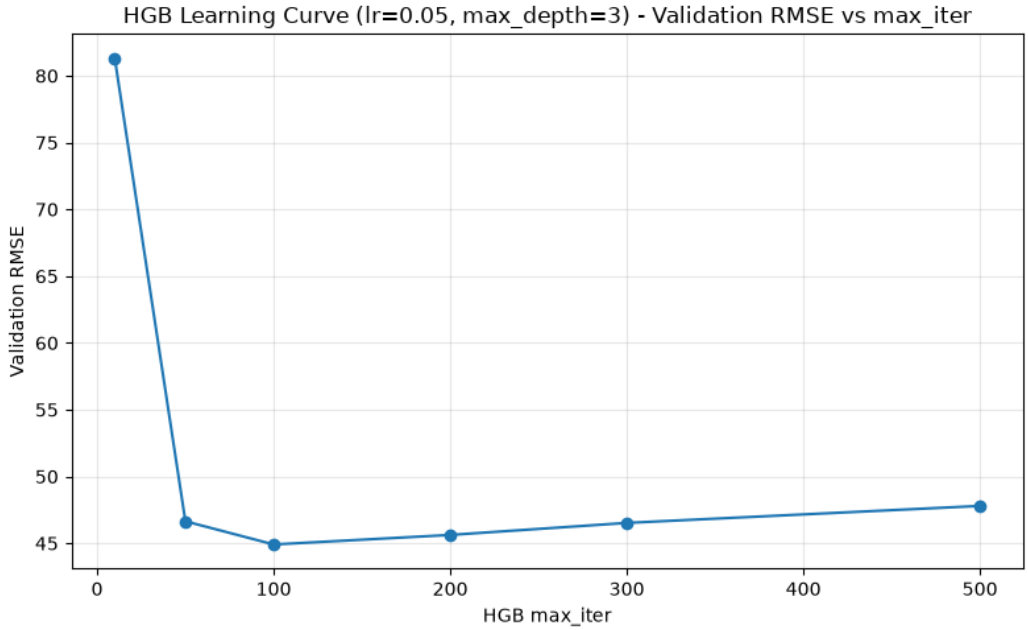


Figure 14: HGB regression learning curve used as training diagnostic material.

6 Phase 4: Bias and error analysis

The bias analysis joins the final regression and classification predictions on date and split. It uses 466 joined rows: 244 validation and 222 test. The subgroup table has 47 rows across seven dimensions: AQI category, season, year, weekday versus weekend, Islamic events, public holidays, and PM2.5 severity band. Missing subgroup keys were counted per split and dimension; all counts were zero in the final join.

The joined validation rows contain 25 scorable Ramadan rows and no scorable Eid rows. The feature table flags Eid on 10–12 April and 17–19 June 2024, but the AQI label is missing on those dates; all six dates also lack a next-day target, so they never enter the split. This is a label-coverage limitation, not a calendar-flag gap. The test split has only one Islamic-event group, `Normal`; no test Ramadan or Eid comparison is therefore possible. Public-holiday test rows number only seven, so the holiday result is marked as small-support. The analysis is environmental and temporal, not demographic: the dataset has one monitoring station and no demographic attributes. It cannot establish fairness across people or neighbourhoods.

Table 13: Missing subgroup keys after joining regression and classification predictions.

Dimension	Validation missing	Test missing
AQI Category	0	0
Islamic_Event	0	0
Public_Holiday	0	0
Season	0	0
Severity_Band	0	0
Weekday_Weekend	0	0
Year	0	0

Table 14 reports the test subgroups. Positive signed error is $y_{true} - y_{pred}$, so a positive value means under-prediction relative to the observed PM2.5. The most important pattern is the severe-pollution tail. Hazardous 401–500 has RMSE 162.14 and signed error 141.70; the 301–500 severity band has RMSE 99.60 and signed error 75.72; and the 500+ band, with only five rows, has RMSE 242.00 and signed error 230.76. These are large errors, not just noise, although the 500+ estimate is unstable because of its tiny support. Severity bands and AQI-category groups are defined by the observed outcome, so high-severity groups look under-predicted and low-severity groups over-predicted partly by construction. The size of the errors (for example RMSE 99.60 on the 301–500 band), not the sign, is the informative result.

Seasonal errors are also uneven. Winter test RMSE is 72.38 with signed error −17.43, indicating average over-prediction, while autumn RMSE is 75.32 with signed error +14.15, indicating average under-prediction. Summer is much easier (RMSE 19.65). Test 2025 has only 45 rows (1 January–16 February, winter), whereas 2024 has 177 mixed-season rows. The 2025 comparison is therefore confounded with season: RMSE is 69.86 versus 63.60, and macro F1 is 0.4306 versus 0.4832, but this is not a clean temporal comparison. Weekday RMSE is higher than weekend RMSE (68.32 versus 55.64), though weekend macro F1 is lower (0.4276 versus 0.4875).

Table 14: Test-set subgroup error and classification performance. Positive signed error means the true value is above the prediction (under-prediction). Small-support groups are marked with *.

Dimension	Group	n	RMSE	MAE	Signed err.	Acc.	Macro F1
AQI Category	Good/Moderate*	8	25.4601	23.6583	-23.6583	0.3750	0.5455
AQI Category	Unhealthy for Sensitive	34	12.5892	9.8540	-7.1164	0.4706	0.6400
AQI Category	Unhealthy	77	43.8019	27.9535	-11.1751	0.7013	0.8244
AQI Category	Very Unhealthy	60	59.1430	44.7748	-27.9941	0.7333	0.8462
AQI Category	Hazardous (301-400)*	24	53.2085	41.5136	11.8486	0.3750	0.5455
AQI Category	Hazardous (401-500)*	19	162.1417	141.6998	141.6998	0.2105	0.3478
Season	Winter	76	72.3755	52.6206	-17.4306	0.6053	0.4494
Season	Summer	55	19.6486	15.0855	0.7674	0.5091	0.4763
Season	Autumn	91	75.3234	46.4062	14.1519	0.6154	0.4358
Year	2024	177	63.6013	39.2136	0.7755	0.5763	0.4832
Year	2025	45	69.8649	46.9118	-2.9323	0.6222	0.4306
Weekday_Weekend	Weekday	158	68.3224	41.7848	1.0546	0.6139	0.4875
Weekday_Weekend	Weekend	64	55.6356	38.2786	-2.5206	0.5156	0.4276
Severity_Band	0-50	31	17.6658	14.2902	-12.6398	0.4839	0.5386
Severity_Band	51-100	60	26.2778	15.2874	1.1969	0.6333	0.5302
Severity_Band	101-150*	28	61.8758	47.0172	-34.7032	0.7143	0.8333
Severity_Band	151-200	31	66.9282	51.5369	-42.0886	0.8387	0.9123
Severity_Band	201-300	43	50.0110	38.7876	-8.5917	0.5349	0.5635
Severity_Band	301-500*	24	99.6015	81.4907	75.7243	0.2083	0.2556
Severity_Band	500+*	5	242.0043	230.7646	230.7646	0.6000	0.7500
Islamic_Event	Normal	222	64.9198	40.7740	0.0239	0.5856	0.4780
Public_Holiday	Normal	215	65.6586	41.4275	-0.3726	0.5767	0.4680
Public_Holiday	Holiday*	7	35.3757	20.7040	12.2017	0.8571	0.7333

Table note: Subgroup macro F1 is computed within each subgroup over the true classes present, so it is not comparable across groups or with Table 9. For AQI Category rows, use accuracy (= recall).

The full set of 14 bias charts (seven dimensions for validation and test) is included in the `figures/` folder. The charts mark groups with $n < 30$ with an asterisk; those estimates should not be read as precise.

The class mix also shifts across periods. Unhealthy is about 48% of training rows but 32% of

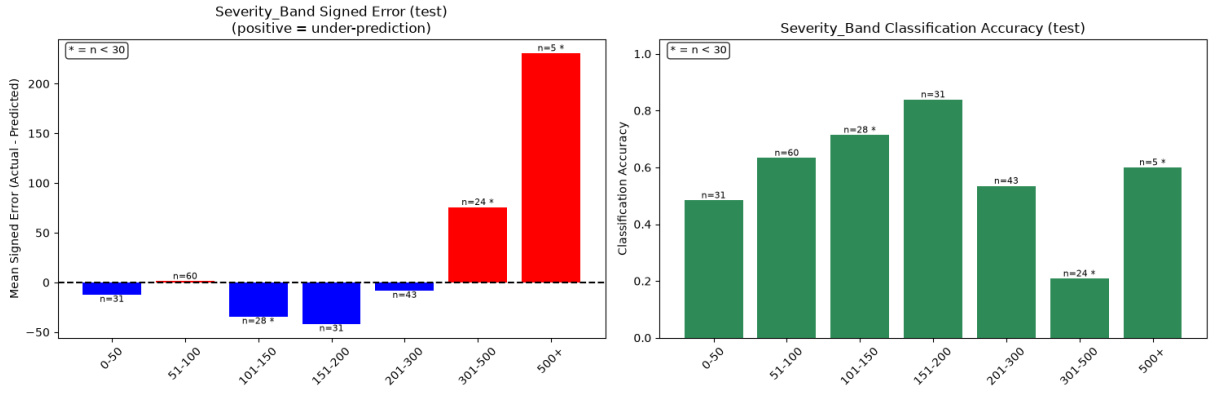


Figure 15: Test signed error and classification accuracy by PM2.5 severity band. Because groups are defined by the observed outcome, the sign is partly constructed; the error magnitude is the informative result. Small groups are marked with an asterisk.

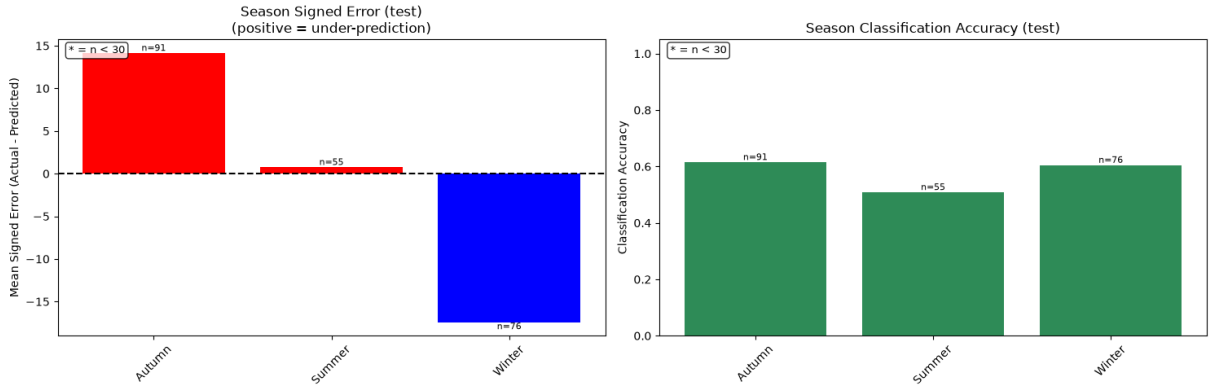


Figure 16: Test signed error and classification accuracy by season.

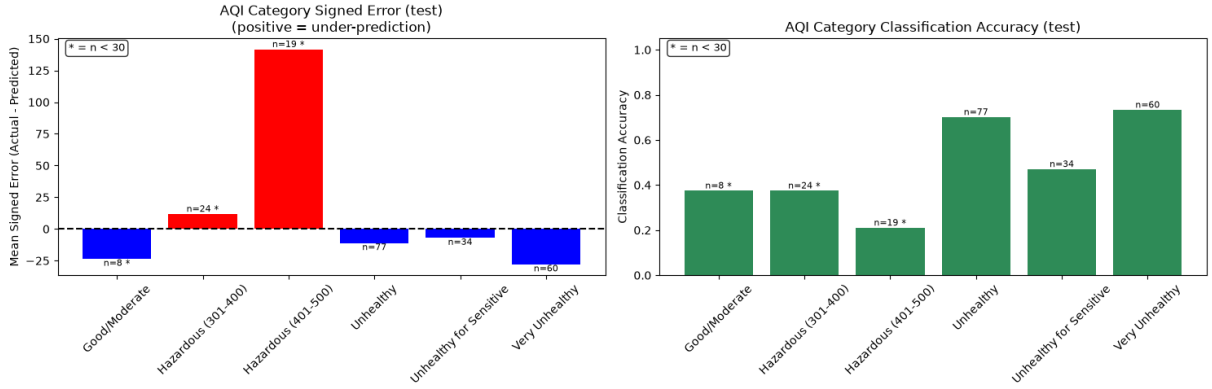


Figure 17: Test signed error and classification accuracy by observed AQI category.

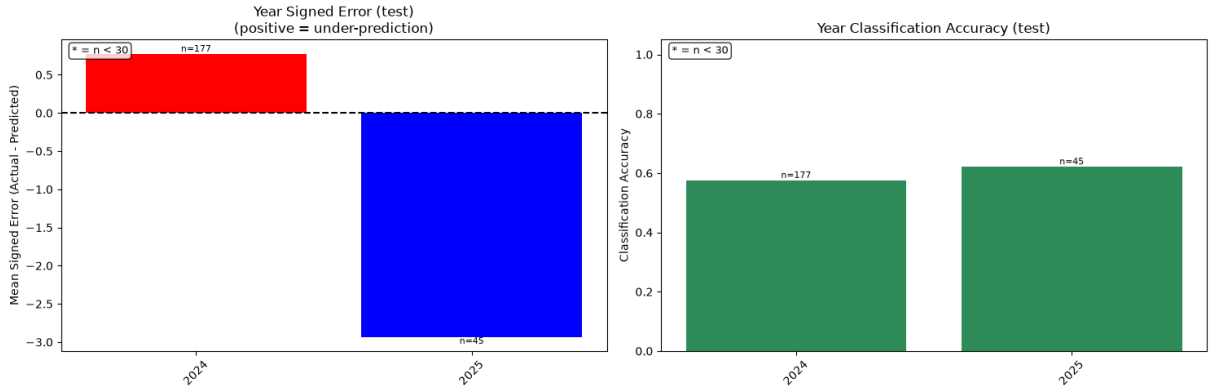


Figure 18: Test signed error and classification accuracy by year.

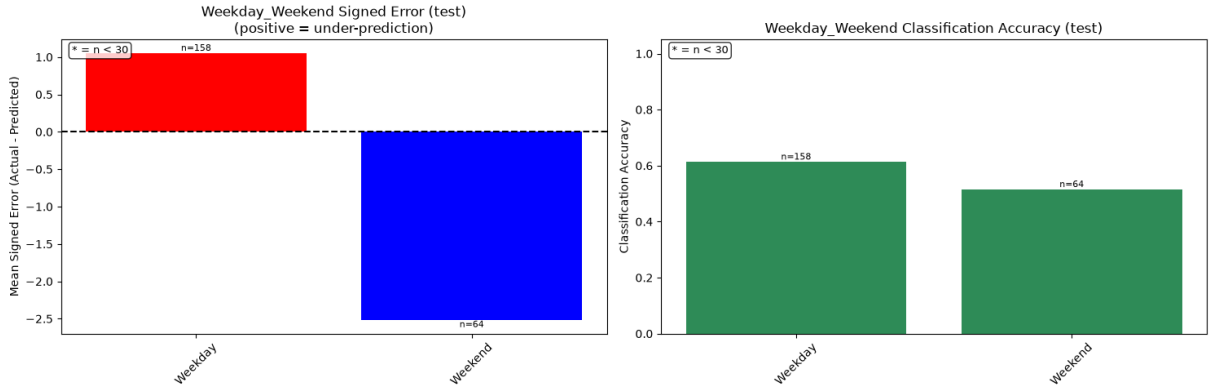


Figure 19: Test signed error and classification accuracy for weekday versus weekend rows.

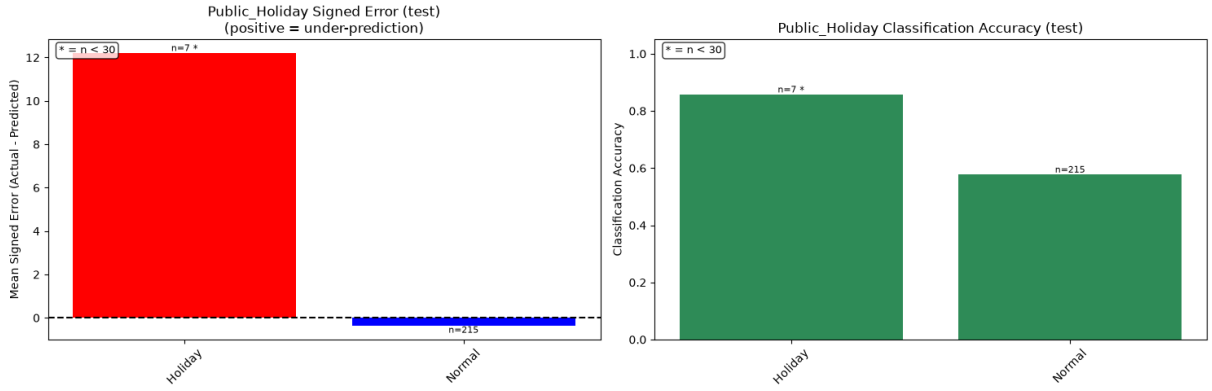


Figure 20: Test signed error and classification accuracy for normal days versus public holidays. Holiday support is only seven rows.

validation and 35% of test. The two Hazardous bands together are about 10% of training but 22% of validation and 19% of test. This may reflect seasonal composition, missing training days or targets, or real worsening; the data do not identify a single cause. The later-period under-prediction of severe days is consistent with that shift, but it is not proof of a causal trend.

7 Phase 4: Model limitations and recommendations

- **Rare-class uncertainty.** Good/Moderate has 8 test rows and Hazardous 401–500 has 19. Their per-class precision/recall are noisy. The same applies to small bias subgroups such as public holidays and the 500+ severity band.
- **Single station.** The dataset represents one monitoring location and has no demographic attributes. The bias check cannot support claims about population-level fairness.
- **Distribution shift.** The class mix and the validation-to-test error pattern change over time. A model selected on validation can fail to beat persistence on a later period, as happened on both tasks (macro F1 below persistence for classification, and essentially tied on RMSE for regression).
- **Coverage gaps.** The validation split contains 250 usable rows over the 326 days from 2023-07-14 to 2024-06-03 (about 77%), because many days lack an AQI label or a next-day target; the test split has 223 rows over the 224 days from 2024-07-08 to 2025-02-16 and contains no spring rows. Validation and test therefore cover different seasonal mixes, which limits how directly their metrics can be compared.
- **Macro F1 versus accuracy.** HGB's test accuracy is higher than persistence's, but its macro F1 is lower. Reporting only accuracy would obscure the rare-class weakness.
- **No traffic data.** Calendar features are proxies; no open, time-aligned Lahore traffic dataset was available. This limits the explanatory scope of the features.
- **No test-time retuning.** The report preserves the validation-based selection. Switching to Ridge after seeing test would be post-hoc and would invalidate the test-once rule.

Recommended next steps are to collect more severe-pollution days, evaluate cost-sensitive or class-balanced variants with a pre-registered validation rule, add calibrated probabilities and prediction intervals, and test on a later period or a second station. Any such change should be selected without using the current test set.

8 Phase 4: Reproducibility

The final results were produced from merge commit `a55a7f5` on `main`; the repository README was updated afterwards (commit `56dea73`). The final pipeline was run from the repository root with:

```
python -m src.check_setup
python -m unittest discover -s tests -t .
python -m src.classification
python -m src.regression
python -m src.bias_analysis
```

`check_setup` reported 0 bad next-day PM2.5 values, 0 bad lag-1 values, and 0 bad next-day categories. The unit tests ran 15 tests and passed. Classification completed in 427.92 seconds and regression in 37.61 seconds. The bias analysis then joined the final prediction files.

The classification prediction file has 473 rows and 12 columns; 466 rows are scorable. Its SHA-256 hash is `47D8E0A89FC3F37EA6B74E807DCF566FBDD6D79F734D5A05CFD56124888EB2EB`, matching the verified Drive copy. Regression also has 473 prediction rows, 466 scorable.

Dependencies are listed in `requirements.txt`. The final environment used pandas 3.0.3, scikit-learn 1.9.1, and matplotlib 3.11.2. The seed is 42 from `configs/base.yaml`. Generated prediction CSVs, metrics JSON, and charts are review artifacts and should not be committed to Git; the data and large artifacts belong on the shared Drive or a release link. No fitted model weights are published: the selected pipelines are refit from fixed seeds by the committed scripts (see the repository README).

8.1 Git-author contributions

Table 15 is the Git-author contribution table: it maps each commit author to the project modules and representative commits they own. The repository history contains only the three project contributors. The Git identities used in the D2 work are Abdul Rehman <arehman4275@gmail.com>, Muhammad Haider Mughal <haiderirfanmm@gmail.com>, and Muhammad Ayub Butt <purestudy83010@gmail.com>. Some earlier commits by the same author also appear under the display name "Haider Mughal" with the same e-mail address. No AI or OpenCode co-author trailer was added.

Table 15: Git-author contribution table: commit authors mapped to project modules. The package and report list exactly the three human project contributors.

Contributor	ID	D2 contribution	Key modules and representative commits
Abdul Rehman	3059	Shared data and evaluation foundation; integration; D2 report	`src/data.py`; `src/evaluate.py`; `src/check_setup.py`; `configs/base.yaml`; `tests/test_evaluate.py`; `reports/` (`b672bfc`, `20fa79e`, `a55a7f5`)
Muhammad Haider Mughal	3006	Classification pipeline and tuning	`src/classification.py` (`d311f8e`, `5f57987`)
Muhammad Ayub Butt	3076	Regression and bias analysis	`src/regression.py`; `src/bias_analysis.py` (`fae7490`, `6e2448d`)

9 Phase 4: Evaluation summary

The primary classification model, HGB, was selected on validation macro F1 and reached 0.4780 on test. It did not beat persistence's 0.5157 macro F1, even though its accuracy was slightly higher. The secondary regression model, HGB, was selected on validation RMSE only; test results were not used to change the selection. Its test RMSE was 64.9198, essentially tied with persistence's 64.8175 (a 0.10 gap and skill −0.0016). Ridge did beat persistence on test with RMSE 62.0956, but switching to it after seeing test would violate the validation-only selection rule. The bias analysis shows that the largest error magnitudes occur on the most severe pollution days; because those groups are defined by the observed outcome, the sign is partly constructed, so the magnitude is the informative result. Seasonal and temporal errors are also uneven. These results are reported as observed, without retuning on the test set.

10 Phase 5: Application and Explainability

Deliverable 3 (Track A) turns the frozen Deliverable 2 pipelines into a working, explainable application. It does not retrain, retune, or reselect a model. The regression and classification pipelines, their preprocessing, the chronological split, and the reported test scores are exactly those of Deliverable 2. The new contribution is the application and inference layer: a six-input forecast screen, an 80% prediction interval, local and global explanations, historical seasonal context, a persistence comparison, and a reproducible artifact pipeline.

This distinction matters for the evaluation. The application cannot repair a model that ties or loses to persistence. It can, however, make the forecast usable, show its uncertainty, expose its limitations, and prevent the user from mistaking a demonstration for an operational warning system.

10.1 Design goals and scope

Table 16 maps the Track A requirements to the implementation and to the evidence that the requirement was met. The design goals were deliberately narrow: a working local application, transparent behaviour, and an honest comparison with the simplest baseline.

Design goal	Implementation	Evidence
Next-day $PM_{2.5}$ and AQI forecast	Frozen HGB regression and six-class HGB classifier in `src/inference.py`; the app shows both views	Regression test RMSE 64.92; classifier test macro-F1 0.478
Uncertainty	Quantile HGB models at the 10th and 90th percentiles, with a validation-calibrated 80% central interval	Validation coverage 80.3%; test coverage 77.0%
Explanations	One-at-a-time local sensitivity and validation-set permutation importance	Sensitivity chart and top-feature panel in the app
Historical context	Monthly medians, winter stagnation, seasonal means, and an hourly profile	Metadata generated by `src/train_pipeline.py` and `src/hourly.py`
Honest baseline comparison	Persistence is displayed on every forecast; the app warns when the two category views disagree	Test metrics and the persistence card are shown in the result panel
Reproducibility	Frozen hyperparameters, fixed seed, refit script, 14 tests, and no committed binaries	The D3 refit reproduces the D2 metrics exactly
Documentation	`README.md`, `STUDY_REPORT.md`, this report, and the report build script	The repository is self-contained apart from the external data files

Table 16: Deliverable 3 design goals and evidence. The model itself is unchanged from Deliverable 2.

Scope. The application is an *illustrative single-day forecaster*, not a live feed. The user supplies six values for today; the app returns a forecast for the following day. It does not ingest a live weather feed, a live sensor stream, or a multi-day history. The remaining model inputs use training-set medians or modes, and the $PM_{2.5}$ lags and rolling windows are held at today's value. Section 10.3 explains why this is a deliberate usability trade-off and where it can mislead.

10.2 System architecture and artifact policy

Figure 21 shows the six layers of the delivered system. The committed data layer contains the source scripts and documentation; the processed feature table is derived and is not committed. The model

layer refits the frozen pipelines and writes four joblib artifacts plus a metadata file. The pipelines use scikit-learn's histogram gradient boosting [6, 7]. The application layer loads those artifacts and performs feature assembly, prediction, interval construction, explanations, and guidance. The interface layer is the Streamlit screen [8]. The evaluation layer contains the ablation and the unit tests. The user outputs are the forecast cards, interval, guidance, and charts.

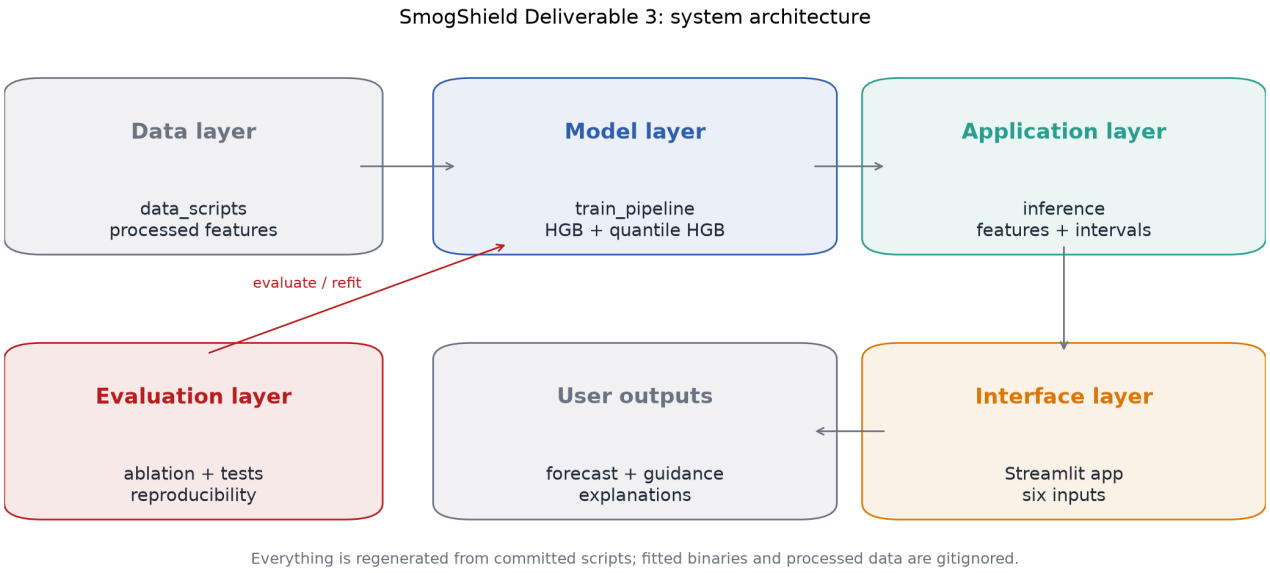


Figure 21: Deliverable 3 system architecture. Fitted artifacts and processed data are regenerated from committed scripts; they are not committed to Git.

Table 17 lists the runtime artifacts. The important policy is that no fitted model binary and no processed data file is committed. The repository contains the scripts that recreate them from fixed seeds. This is the same policy used in Deliverable 2, extended to the four new pipelines and the metadata file.

Artifact	Produced by	Consumed by	Committed?
`models/regression_pipeline.joblib`	`src/train_pipeline.py`	`src/inference.py`	No
`models/regression_q10_pipeline.joblib`	`src/train_pipeline.py`	`src/inference.py`	No
`models/regression_q90_pipeline.joblib`	`src/train_pipeline.py`	`src/inference.py`	No
`models/classification_pipeline.joblib`	`src/train_pipeline.py`	`src/inference.py`	No
`models/metadata.json`	`src/train_pipeline.py`	`src/inference.py`, app	No
`models/ablation.json`	`src/ablation.py`	report, review	No
`data/processed/features_daily.csv`	`data_scripts/09_build_features.py`	`src/train_pipeline.py`	No
`data/processed/pm25_hourly.csv`	`data_scripts/02_clean_pm25_simple.py`	`src/hourly.py`	No

Table 17: Runtime artifacts and their Git policy. The model directory has its own `.gitignore`; the processed data directory is ignored at the repository root.

The architecture separates *training* from *serving*. The Streamlit app imports `src.inference`, not

the training modules. The inference module loads the pipelines once, caches them in memory, and exposes a single `predict()` call. If the artifacts are missing, it raises an error that points the user at `python -m src.train_pipeline`. This makes the failure mode explicit rather than silently serving a stale or partial model.

10.3 From six inputs to 46 features

The model contract contains 46 raw features, which become 80 encoded columns after one-hot encoding and missing-value indicators. The app collects only six user-facing inputs. Table 18 shows how each input is mapped onto the model's real feature names.

App input	Unit	Derived model features	Default
Feature date	date	`month`, `day_of_year`, `weekday`, `is_weekend`, sin/cos terms, season dummies	Today's date
Today's $PM_{2.5}$	$\mu g/m^3$	`pm25`, `pm25_log`, numeric `aqi`, lags 1–3, rolling 3/7, difference	Monthly historical median
Today's AQI category	seven raw labels	`aqi_category` (one-hot encoded)	Derived from the default $PM_{2.5}$
Temperature	°C	`temp_mean_c`, `temp_max_c`, `temp_min_c`, `temp_mean_c_lag1`	22
Relative humidity	%	`humidity_mean_pct`, `humidity_mean_pct_lag1`	60
Wind speed	km/h	`wind_speed_mean_kmh`, `wind_speed_max_kmh`, `wind_speed_mean_kmh_lag1`	7

Table 18: The six user inputs and the model features they populate. Unlisted features use training-set medians or modes.

The feature contract is filled in four steps. First, `build_feature_row` starts from the training medians for numeric features and the training modes for categorical features, both stored in `metadata.json`. Second, it maps the six friendly inputs onto the model columns. Third, it derives the $PM_{2.5}$ history as a steady state: lags 1–3, the 3-day and 7-day rolling means, and the day-to-day difference are set from today's $PM_{2.5}$, with the difference set to zero. Fourth, explicit feature-column overrides are applied last. The date-derived calendar features are computed from the selected feature date, not from the forecast date; the forecast is always for the following day.

The numeric AQI fix. The app does not ask for a numeric AQI. In an early version the `aqi` feature therefore stayed at the training median of 165 even when the user entered a clean or hazardous $PM_{2.5}$ value. That was internally inconsistent: the model saw a category and a $PM_{2.5}$ value that did not correspond to the same AQI. The inference layer now derives `aqi` from the entered $PM_{2.5}$ through the same US-EPA breakpoints used to build the training labels. This is an inference-layer fix; the fitted pipelines and all reported metrics are unchanged.

The category input is nearly inert. The raw AQI category is a categorical feature, but the numeric AQI and $PM_{2.5}$ carry almost all of the signal. In an audit of the default November example, changing the category input across all seven raw labels left the regression prediction identical to two decimals (189.74 $\mu g/m^3$). The classifier's top probability moved only between 0.9865 and 0.9910, and its arg-max stayed *Very Unhealthy*. This is not a bug in the UI; it is a property of the fitted model.

The app keeps the category input because it is part of the model contract and because the raw seven-label vocabulary is familiar to users, but the report does not claim that the category selector drives the forecast.

Input category	Regression $PM_{2.5}$	Classifier top label (probability)
Good	189.74	Very Unhealthy (0.991)
Moderate	189.74	Very Unhealthy (0.991)
Unhealthy for Sensitive	189.74	Very Unhealthy (0.991)
Unhealthy	189.74	Very Unhealthy (0.987)
Very Unhealthy	189.74	Very Unhealthy (0.991)
Hazardous (301–400)	189.74	Very Unhealthy (0.991)
Hazardous (401–500)	189.74	Very Unhealthy (0.991)

Table 19: Category-input audit for the default November example ($PM_{2.5} = 234.2$, temperature 22°C, humidity 60%, wind 7 km/h, 12 November 2026). The numeric $PM_{2.5}$ and derived AQI dominate; the category selector is nearly inert.

Feature groups. Table 20 groups the 46 raw features. The 12 history features are the inputs that the ablation calls "history only." Adding 17 calendar features gives the 29-feature set; adding 17 weather features gives the full 46-feature set. This is the same nested design used in the Deliverable 3 ablation.

Feature group	Raw count	Examples
Air quality and history	12	`pm25`, `aqi`, `aqi_category`, `pm25_log`, lags 1–3, rolling 3/7, difference, data-quality flags
Calendar and events	17	month, day-of-year, weekday, weekend, sin/cos terms, season dummies, Ramadan, Eid, public holiday
Weather	17	temperature (mean/max/min/lag), humidity (mean/lag), dew point, pressure (mean/lag), wind (mean/max/lag/direction), rain, solar radiation, cloud cover
Total	46	80 encoded columns after preprocessing

Table 20: The nested feature groups. The ablation uses the 12-, 29-, and 46-feature sets with the same frozen hyperparameters.

10.4 The forecast screen

Figure 22 shows the flow of the forecast screen. The user enters the six values in a two-column form. Pressing *Forecast next day* calls `inference.predict()` and returns three rows of output. The first row shows the predicted $PM_{2.5}$, the AQI implied by that $PM_{2.5}$, and the implied AQI category. The second row shows the classifier's category, its confidence, and the persistence value. The third block shows the 80% interval, the guidance banner, the local sensitivity chart, and the expanders for global importance, winter stagnation, and the hourly profile.

Table 21 explains each output. The table is intentionally redundant with the app: a user should not have to infer what a card means, and a reviewer should be able to check that the screen is not hiding a weaker result.

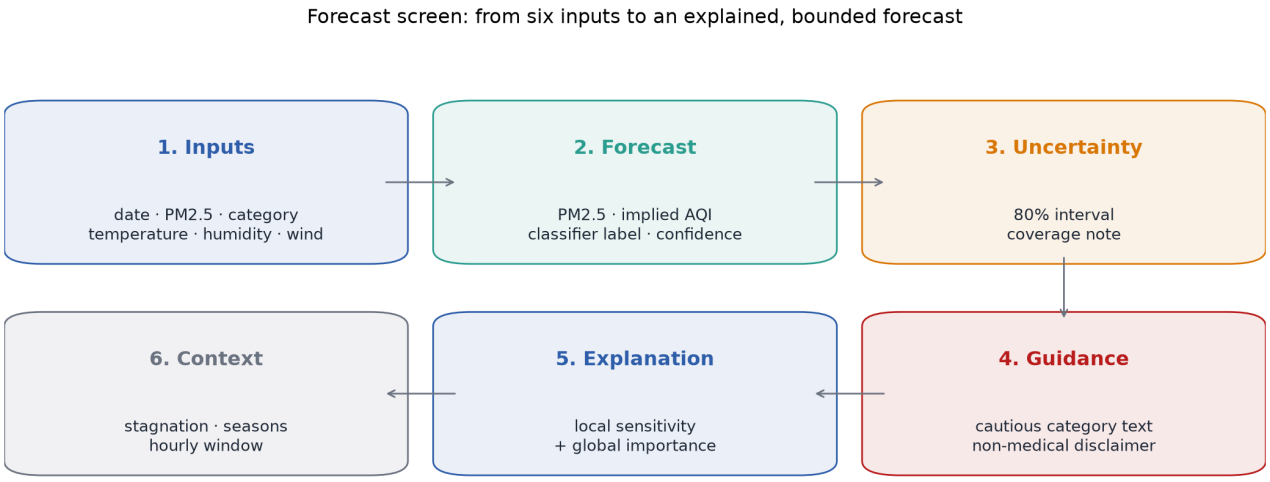


Figure 22: The forecast screen in six steps. The interface is a single Streamlit page; the expanders keep the historical context available without crowding the primary forecast.

Output	What it shows	How to read it
Predicted $PM_{2.5}$	Regression point forecast for tomorrow	A single number in $\mu g/m^3$; not a guarantee
AQI implied by $PM_{2.5}$	US-EPA AQI computed from the predicted $PM_{2.5}$	Monotonic in the prediction; uses the same breakpoints as the labels
Implied category	EPA category of the implied AQI	The more defensible category view because the classifier was weaker than persistence
Classifier category and confidence	Six-class HGB arg-max and its probability	A separate model; biased toward *Unhealthy*
Persistence	Tomorrow = today, for $PM_{2.5}$ and category	The baseline the model must beat; shown on every forecast
80% interval	Calibrated central interval for next-day $PM_{2.5}$	A range, not a worst case; test coverage is 77.0%
Local sensitivity	Change in the forecast when one input is increased	A finite difference, not a causal effect
Global importance	Permutation importance on validation	Which features the model relies on overall
Winter stagnation	Share of stagnant winter days and their mean $PM_{2.5}$	Descriptive history, not a today-specific forecast
Best time outdoors	Hour-of-day and seasonal $PM_{2.5}$ profile	Station-specific history; winter afternoons remain polluted
Guidance	Cautious, non-medical advice for the implied category	Follow official advisories; not medical advice
Disagreement warning	Appears when the two category views differ	Explains which view the app recommends and why

Table 21: The forecast screen outputs and their interpretation.

The interface was exercised with Streamlit's application test harness. The AppTest run reported no exceptions, four charts, and three expanders. This is a smoke test, not a usability study: it confirms that the page renders and the main branches execute, not that the wording is clear to a first-time user.

10.5 Prediction intervals

A point forecast is not enough for a decision. The app therefore trains two additional HGB models with quantile loss, one at the 10th percentile and one at the 90th percentile, and combines them into an 80% central prediction interval. The bounds are forced to contain the point forecast and are then widened by a calibration factor chosen on the validation split. On this dataset the factor is 1.00, so no widening was needed. Validation coverage is 80.3%; test coverage is 77.0%. The test split is used only to report coverage, never to choose the factor.

Property	Value
Nominal level	80%
Quantiles	10th and 90th
Calibration scale	1.00
Validation coverage	80.3%
Test coverage	77.0%
Tuning rule	Validation only; test reported, never tuned

Table 22: The 80% prediction interval. The test coverage is below nominal, which is reported rather than hidden.

Figure 23 shows the nominal-versus-empirical coverage and the default November example. The default is deliberately a polluted day: today's $PM_{2.5}$ is the historical November median, 234.2 $\mu g/m^3$; the app forecasts 189.74 $\mu g/m^3$ with an interval of 133.8–294.1 $\mu g/m^3$. The interval spans categories from *Unhealthy* to *Hazardous (301–400)*, even though the point forecast is *Very Unhealthy*. That width is the honest message: the model is not precise enough to support a single category claim.

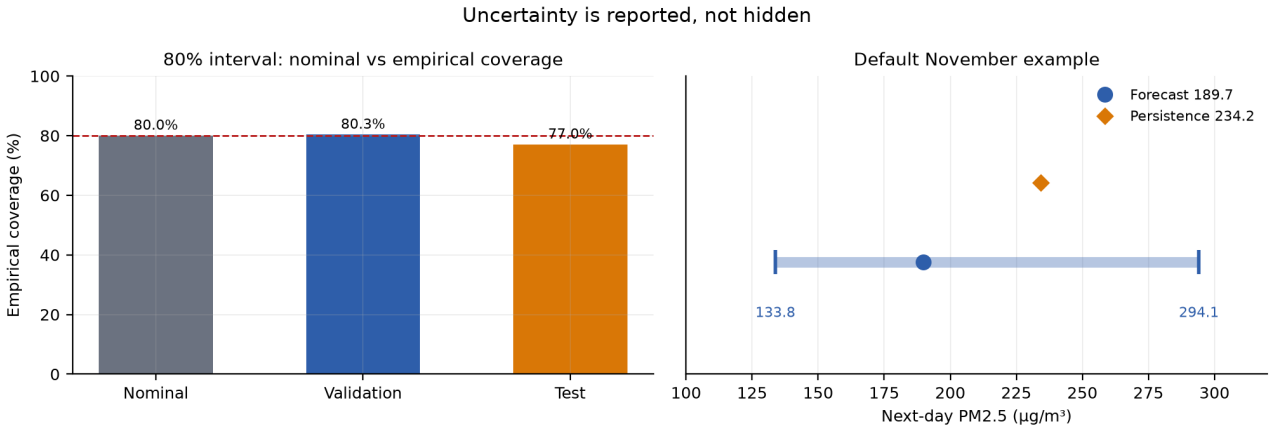


Figure 23: Left: nominal 80% coverage versus validation and test coverage. Right: the default November forecast and its 80% interval, compared with persistence. The interval is wide because the model is uncertain, not because the app is hiding a point estimate.

The interval is a central interval for the *next-day $PM_{2.5}$ observation*, not a confidence interval for the model's mean prediction. It does not include model-selection uncertainty, distribution shift, or

the error introduced by the steady-state feature assumption. A future version should add conformal prediction and a separate interval for the AQI category.

10.6 Explanations: global and local

The app provides two explanations with different scopes. The global explanation uses permutation importance on the validation split: one feature at a time is shuffled, and the increase in RMSE (regression) or the drop in macro-F1 (classification) is recorded. The local explanation perturbs one input for the current forecast and reports the change in the predicted $PM_{2.5}$.

Global view. Figure 24 and Table 23 show the top features. For regression, today's $PM_{2.5}$ is dominant (increase in RMSE 14.51), followed by the numeric AQI (4.20), day-of-year (4.15), lag-1 $PM_{2.5}$ (2.93), and wind direction (2.50). For classification, the top features are $PM_{2.5}$ (drop in macro-F1 0.0786), wind direction (0.0672), pressure (0.0506), numeric AQI (0.0495), and the day-of-year cosine (0.0407). The presence of day-of-year and seasonal terms is consistent with the strong winter cycle.

The ranking is descriptive, not causal. Permutation importance breaks the relationship between a feature and its correlated neighbours. Shuffling `pm25` while leaving the lags intact can overstate its unique contribution; conversely, a feature that is redundant with another may look unimportant. The report uses the ranking to explain what the model relies on, not to claim that changing a feature would change the outcome by that amount.

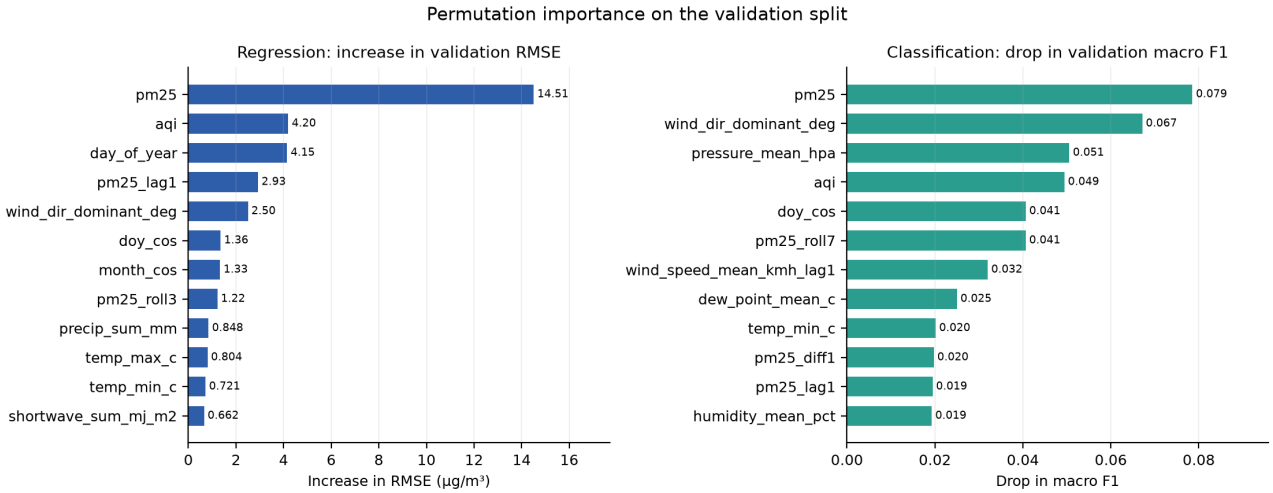


Figure 24: Permutation importance on the validation split. Regression scores are the increase in RMSE when a feature is shuffled; classification scores are the drop in macro-F1. Correlated features share credit, so the ranking is not a causal decomposition.

Local view. The local explanation answers a narrower question: for *this* forecast, what happens if one input is increased by a fixed amount? The app perturbs four inputs: $PM_{2.5}$ by +25 $\mu g/m^3$, temperature by +5°C, humidity by +10 percentage points, and wind speed by +5 km/h. The $PM_{2.5}$ step is larger than the others because HGB predictions are piecewise constant: a +10 step often stays inside the same leaf and reports a misleading 0.0. A +25 step crosses splits more reliably, but it is still a finite difference, not a smooth slope.

Table 24 shows the local explanation for the default November example. A +25 $\mu g/m^3$ increase in today's $PM_{2.5}$ raises the forecast by 38.68 $\mu g/m^3$; a +5°C temperature increase raises it by 18.41;

Regression (increase in RMSE)			Classification (drop in macro-F1)		
Rank	**Feature**	**Value**	**Rank**	**Feature**	**Value**
1	pm25	14.51	1	pm25	0.0786
2	aqi	4.20	2	wind_dir_dominant_deg	0.0672
3	day_of_year	4.15	3	pressure_mean_hpa	0.0506
4	pm25_lag1	2.93	4	aqi	0.0495
5	wind_dir_dominant_deg	2.50	5	doy_cos	0.0407
6	doy_cos	1.36	6	pm25_roll7	0.0406
7	month_cos	1.33	7	wind_speed_mean_kmh_lag1	0.0320
8	pm25_roll3	1.22	8	dew_point_mean_c	0.0250

Table 23: Top eight permutation-importance features for the two tasks.

a +5 km/h wind increase lowers it by 7.08; and a +10-point humidity increase lowers it by 0.43. The wind sign matches the physical intuition that stronger wind disperses pollution. The temperature sign is local and counter-intuitive at first glance; it should not be read as a causal effect, because temperature is correlated with season, pressure, and the winter inversion regime.

Input change	**Change in forecast**	**Direction**
$PM_{2.5}$ +25 $\mu g/m^3$	+38.68	Higher forecast
Temperature +5°C	+18.41	Higher forecast
Wind +5 km/h	−7.08	Lower forecast
Humidity +10 points	−0.43	Slightly lower forecast

Table 24: Local sensitivity for the default November example (point forecast 189.74 $\mu g/m^3$). The values are one-at-a-time finite differences, not causal effects.

The app ranks the four bars by absolute effect. A zero-length bar is possible even for an important feature if the perturbation stays inside a leaf; the larger $PM_{2.5}$ step reduces, but does not eliminate, that artifact. A Shapley-value decomposition would be a more principled local explanation and is left as future work.

10.7 Seasonal defaults and historical context

The app opens on a seasonally plausible scenario rather than a fixed clean-day value. The initial $PM_{2.5}$ is the historical median for the current calendar month, and the initial category is derived from that $PM_{2.5}$ through the EPA breakpoints. On 12 November 2026 the default is 234.2 $\mu g/m^3$ and *Very Unhealthy*; the forecast is 189.74 $\mu g/m^3$, still *Very Unhealthy*, with a persistence value of 234.2 $\mu g/m^3$.

This choice matters for the demo. A fixed 60 $\mu g/m^3$ default would open the app on a moderately polluted day that is not representative of November in Lahore. The monthly medians in Table 25 and Figure 25 show the size of the seasonal swing: the median is 253.3 $\mu g/m^3$ in December, 234.2 in November, and 44.4 in August. The default is a starting point, not a live reading; the user can override it.

Winter stagnation. The app also reports a simple historical pattern. A winter day (December–February) is labelled *stagnant* when its mean wind speed is in the winter lower quartile and it recorded at most 0.5 mm of rain. Of 407 winter days, 100 (24.6%) meet the definition. Their mean $PM_{2.5}$ is 294.9 $\mu g/m^3$, versus 193.7 $\mu g/m^3$ on better-ventilated days, a difference of +52.2%. The winter wind

Month	Median $PM_{2.5}$	Month	Median $PM_{2.5}$
January	216.5	July	47.5
February	145.0	August	44.4
March	83.7	September	62.1
April	59.8	October	119.1
May	70.9	November	234.2
June	67.0	December	253.3

Table 25: Historical monthly median $PM_{2.5}$ ($\mu g/m^3$), computed from physically valid daily values. The app uses the current month's median as its initial $PM_{2.5}$ default.

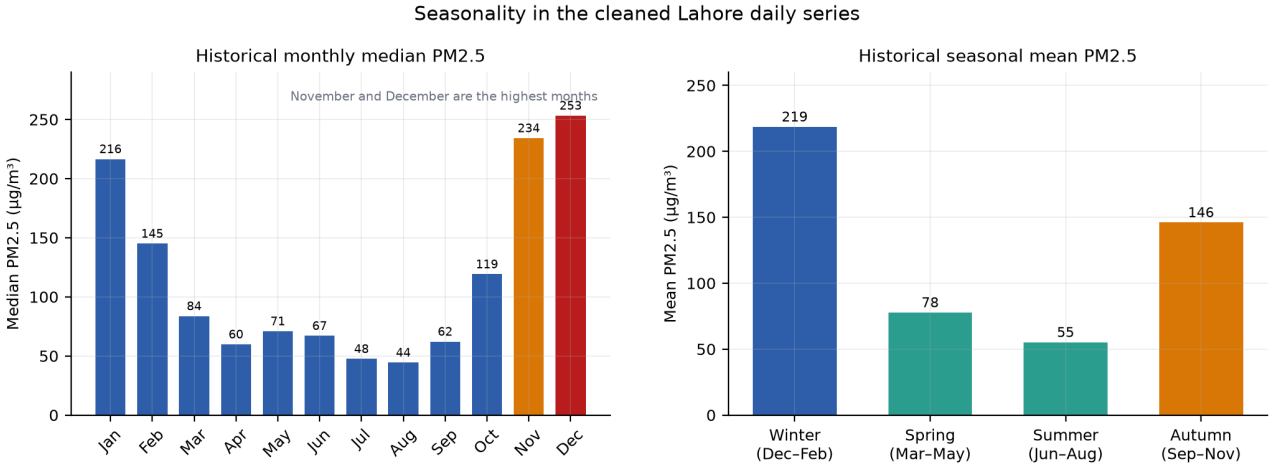


Figure 25: Historical monthly medians and seasonal means. November and December are the highest months; summer is much cleaner. These are descriptive statistics from one station, not a forecast.

quartile threshold is 5.2 km/h. This is a descriptive proxy for stagnation, not a dispersion model, and it should not be read as a causal estimate of the effect of wind.

Quantity	Value
Winter days (Dec–Feb)	407
Stagnant days	100 (24.6%)
Winter wind lower quartile	5.2 km/h
Mean $PM_{2.5}$ on stagnant days	294.9 $\mu g/m^3$
Mean $PM_{2.5}$ on ventilated days	193.7 $\mu g/m^3$
Difference	+52.2%

Table 26: Winter stagnation summary. The definition is descriptive and the difference is associational.

Seasonal means. The seasonal means are 218.6 $\mu g/m^3$ in winter, 146.0 in autumn, 77.7 in spring, and 55.1 in summer. These values are the context for the app's guidance: a November forecast that looks lower than today can still be *Very Unhealthy*, and a winter afternoon that is cleaner than the morning can still be far above the WHO guideline. The app says this explicitly rather than implying that timing solves the problem.

10.8 The hourly profile

The hourly card uses 36,820 valid hourly observations from the Lahore US Diplomatic Post sensor, covering 2019-05-23 to 2025-02-18. The overall cleanest three-hour window is 14:00–16:59, with a mean $PM_{2.5}$ of 85.2 $\mu g/m^3$; the worst hour is 22:00, with 138.4 $\mu g/m^3$. The afternoon minimum holds in every season, but the winter version of the same window still averages 155.4 $\mu g/m^3$, versus 41.2 in spring, 44.4 in summer, and 90.8 in autumn. Timing helps; it does not make winter air safe.

Window or season	Mean $PM_{2.5}$
Cleanest overall window (14:00–16:59)	85.2 $\mu g/m^3$
Worst hour (22:00)	138.4 $\mu g/m^3$
Winter cleanest window (14:00–16:59)	155.4 $\mu g/m^3$
Spring cleanest window	41.2 $\mu g/m^3$
Summer cleanest window	44.4 $\mu g/m^3$
Autumn cleanest window	90.8 $\mu g/m^3$

Table 27: The hourly profile. The source is a single station and the values are descriptive history, not a today-specific hourly forecast.

The card carries a caveat in the app: it is a descriptive station history in Pakistan local time, not a today-specific forecast. The hourly source file is not committed; if it is absent, the app omits the card and the rest of the pipeline still runs.

10.9 Persistence, guidance, and the honesty contract

The app displays persistence on every forecast. Table 28 repeats the comparison that drives the report's framing. On the test split, the regression's RMSE is 64.92 versus 64.82 for persistence, a difference of 0.10 $\mu g/m^3$ and a skill score of −0.0016. The classifier's macro-F1 is 0.4780 versus 0.5157 for persistence, even though the classifier's accuracy is slightly higher (0.5856 versus 0.5676). The

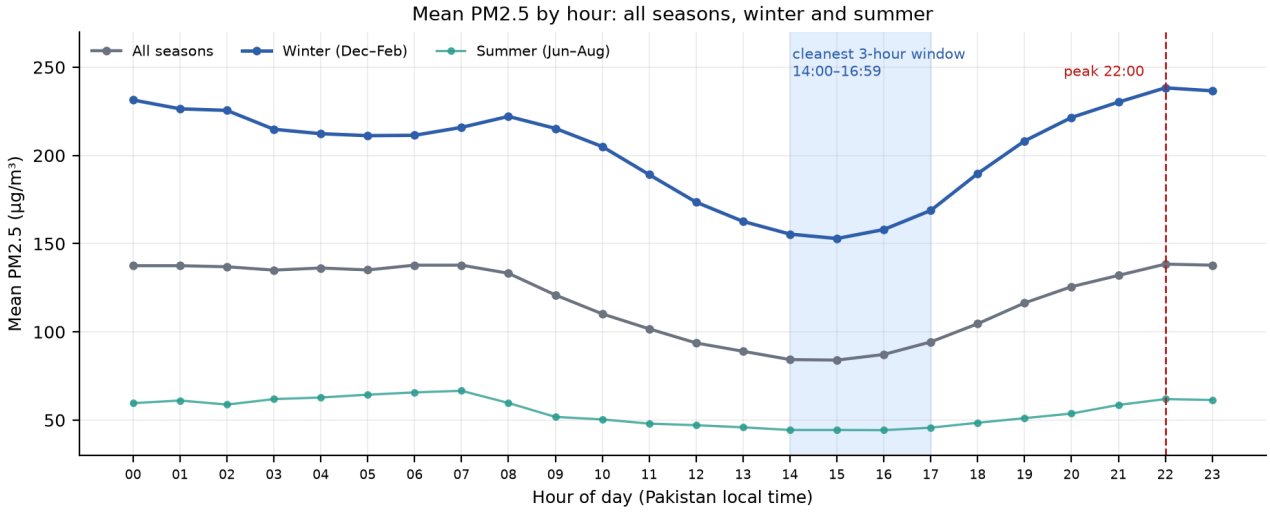


Figure 26: Mean $PM_{2.5}$ by hour for all seasons, winter, and summer. The shaded window is the overall cleanest three hours; the dashed line marks the worst hour. Winter afternoons remain polluted.

accuracy/macro-F1 split is the key: accuracy is dominated by the common *Unhealthy* class, while macro-F1 gives the rare hazardous classes equal weight.

Task and model	Validation	Test	Persistence validation	Persistence test
Regression, HGB RMSE	45.62	64.92	51.55	64.82
Classification, HGB macro-F1	0.4941	0.4780	0.4766	0.5157

Table 28: The frozen model versus persistence. The regression ties persistence on test; the classifier loses on macro-F1. Ridge reached a lower test RMSE (62.10) but was not the validation-selected model, so it is not used.

When the two category views disagree, the app shows a warning and says that the implied category is the more defensible view because the classifier was weaker than persistence on test. This is a deliberate design choice: hiding the disagreement would make the interface look more confident than the evidence supports.

Guidance. The guidance text is keyed to the regression-implied category and is cautious and non-medical. Table 29 lists the headlines. Every message carries a disclaimer that it is not a substitute for official health advice. The app does not tell a user that it is safe to exercise; it tells a user when to consider reducing exertion and when to avoid prolonged outdoor activity.

10.10 Tests and reproducibility

The application layer has 14 unit tests. They cover the EPA breakpoints and category mapping, the feature schema and defaults, the interval bounds, the local explanation, the guidance dictionary, the seasonal/stagnation patterns, the monthly medians, and the hourly profile. The Streamlit AppTest adds a render smoke test: no exceptions, four charts, and three expanders.

The reproducibility check is stronger than the test suite. Refitting the D3 pipelines from the committed script reproduces the Deliverable 2 metrics exactly: regression validation/test RMSE 45.6243/64.9198, persistence 51.5457/64.8175; classification validation/test macro-F1 0.4941/0.4780,

Implied category	Headline
Good/Moderate	Air quality is relatively favourable.
Unhealthy for Sensitive	Sensitive groups should take care.
Unhealthy	Consider reducing outdoor exertion.
Very Unhealthy	Avoid prolonged outdoor activity.
Hazardous (301–400)	Avoid outdoor activity.
Hazardous (401–500)	Avoid outdoor activity.

Table 29: Cautious guidance by implied AQI category. The app adds a non-medical disclaimer to every message.

Test area	What is checked
AQI conversion	Breakpoints and category labels match the training labels
Feature contract	The app builds the expected columns, with the expected defaults
Intervals	Bounds contain the point forecast and respect the calibration factor
Explanation	Sensitivity returns the four perturbations in the expected shape
Guidance	Every category has a headline, detail, and disclaimer
Patterns	Stagnation, seasonal means, monthly medians, and hourly profile are present
App render	AppTest reports no exceptions, four charts, and three expanders

Table 30: Test coverage for the Deliverable 3 application layer.

persistence 0.4766/0.5157; interval coverage 80.3%/77.0%. The D2 code and report were not edited. The D3 training script adds the two classifier settings that were needed to match D2 exactly: balanced class weights and disabled early stopping. The model itself is unchanged.

11 Phase 6: Final Evaluation and Discussion

This section integrates the six phases, evaluates the application as a whole, and states what the project can and cannot claim. The central finding is negative but useful: the frozen model does not beat persistence, and the application is designed to say so.

11.1 Integrated results across phases

Table 31 summarizes the delivered phases. The project moved from a problem definition and a documented dataset, through a validation-selected model and a bias analysis, to an application and an honest final evaluation. The model scores did not change between Deliverable 2 and Deliverable 3.

The contribution is not a new accuracy record. It is an end-to-end, reproducible system that makes a difficult result visible: a strong one-day persistence baseline is hard to beat with the available features, and the remaining errors are concentrated on the most severe pollution days, which are exactly the days that matter most for public health.

11.2 Ablation: do weather and calendar features help?

The ablation refits both tasks on three nested feature sets with the frozen hyperparameters: history only (12 features), history plus calendar (29), and the full set (46). Test metrics are reported but

Phase	Main output	Headline
Phase 1	Problem definition and objectives	1,506 valid days; only 9 Good days (0.6%); 73.2% Unhealthy or worse
Phase 2	Data foundations and EDA	2,113 daily rows, 63 columns, 2019-05-23 to 2025-03-04; one station
Phase 3	Model development and selection	HGB selected on validation: RMSE 45.62, macro-F1 0.4941
Phase 4	Evaluation and bias analysis	Test RMSE 64.92 vs persistence 64.82; macro-F1 0.478 vs 0.516; severe-tail errors are the largest
Phase 5	Application and explainability	Six inputs, two category views, 80% interval, local/global explanations, seasonal defaults, stagnation, hourly profile
Phase 6	Final evaluation and discussion	Ablation, backtest, limitations, reproducibility, and an honest verdict: the app demonstrates the pipeline; it does not beat persistence

Table 31: Integrated results across the six phases.

never used to choose a set. The results are in Table 32 and Figure 27.

Feature set	Feat.	Reg. val	Reg. test	Clf. val	Clf. test
History only	12	52.18	63.88	0.4663	0.4715
History + calendar	29	49.97	65.16	0.4319	0.5155
Full (adds weather)	46	**45.62**	**64.92**	**0.4941**	**0.4780**

Table 32: Nested feature-set ablation. Reg. columns are RMSE (μg/m^3); Clf. columns are macro-F1. Bold marks the validation-selected full model. Test values are reported, not used for selection.

The full model is best on validation for both tasks, which is why it is the delivered model. The test pattern is mixed. Adding calendar features improves classification test macro-F1 from 0.4715 to 0.5155 but lowers regression test RMSE from 63.88 to 65.16. The full model then improves validation but gives back some test performance. The honest conclusion is that there is no consistent held-out gain from adding weather and calendar features on this dataset. The ablation is descriptive, not a second model-selection step; switching to the history-plus-calendar classifier after seeing test would violate the validation-only rule.

11.3 Backtest: 1 November 2024

A single backtest makes the application's limitations concrete. The feature date is 1 November 2024, which is in the test split. Today's $PM_{2.5}$ is 156.6 μg/m^3. The app's six-input forecast is 159.23 μg/m^3, with an 80% interval of 101.7–209.7. The full-feature pipeline, using the actual history and weather for that date, forecasts 188.23. The observed next-day $PM_{2.5}$ is 278.86. Persistence is 156.6. The actual value is outside the app's interval.

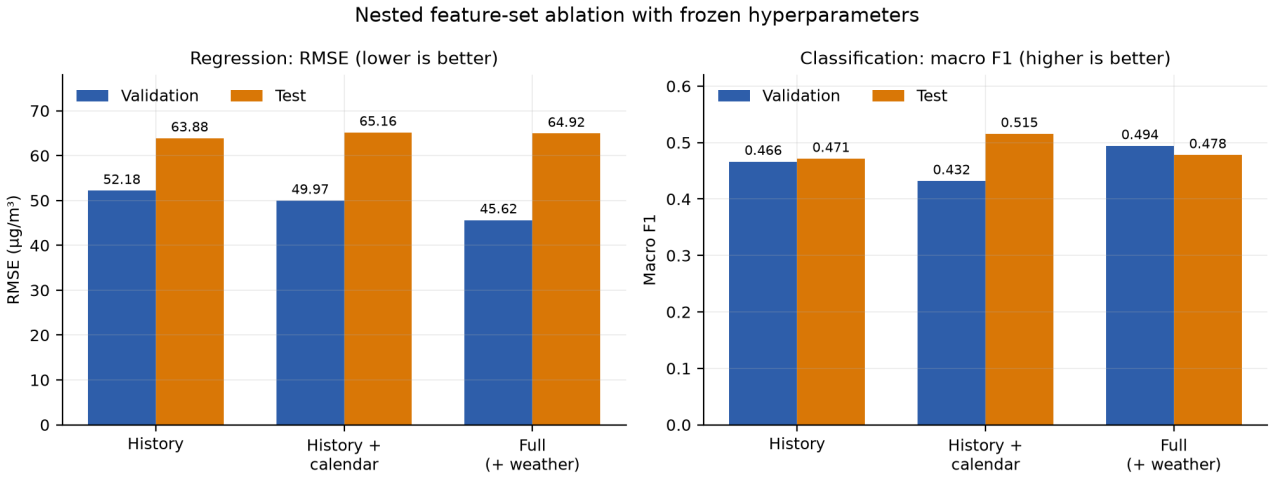


Figure 27: Ablation results. The full model is best on validation for both tasks, but the test pattern is mixed: calendar features help classification and hurt regression, and the full model is not consistently best on test.

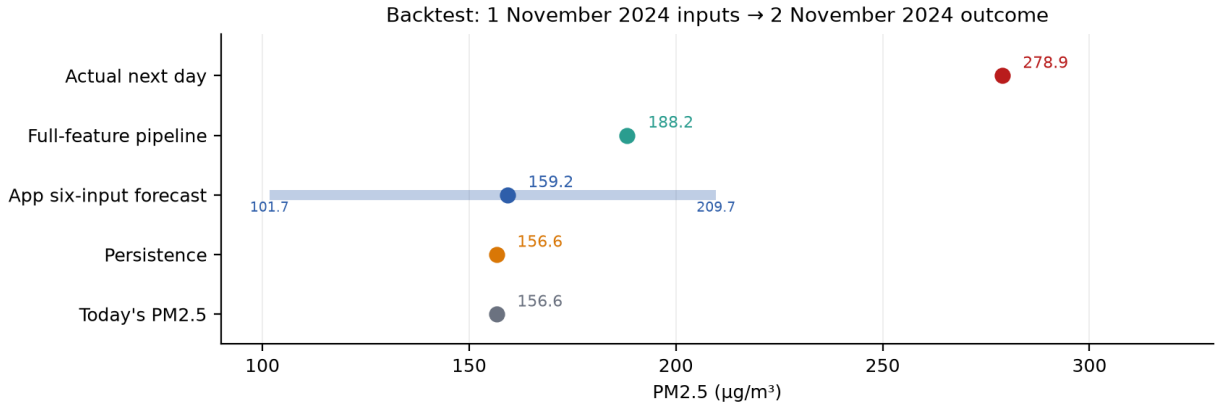


Figure 28: Backtest for 1 November 2024. The app forecast is close to persistence and far below the actual outcome; the full-feature model is closer but still under-predicts. This is the severe-tail failure that the bias analysis identified.

Quantity	Value
Feature date	1 November 2024
Today's $PM_{2.5}$	156.6 $\mu g/m^3$
App six-input forecast	159.23 $\mu g/m^3$
App 80% interval	101.7–209.7 $\mu g/m^3$
Full-feature pipeline forecast	188.23 $\mu g/m^3$
Actual next-day $PM_{2.5}$ (2 November 2024)	278.86 $\mu g/m^3$
Persistence	156.6 $\mu g/m^3$

Table 33: Backtest for 1 November 2024. The app forecast and the full-feature forecast both under-predict the severe day; the app's interval misses the actual value.

The difference between 159.23 and 188.23 is informative. The app does not have the actual previous days' $PM_{2.5}$ or the actual weather, so it assumes a steady state and uses training medians for

uncollected weather features. The full-feature pipeline has the real inputs. Even with those inputs, it under-predicts a severe day by about 91 μg/m^3. The app's lower forecast is not evidence that the simpler interface is better; it is a consequence of different information. The interval is wide, but not wide enough for this outlier. With 77% test coverage, some misses are expected; the important point is that the miss is on a high-severity day, where the cost of under-prediction is largest.

11.4 What the app can and cannot claim

Table 34 separates the defensible claims from the indefensible ones. The app is a demonstration of an end-to-end explainable pipeline. It is not an operational air-quality warning system.

The app can support	**The app cannot support**
A next-day $PM_{2.5}$ and AQI-category forecast from six inputs	A claim that the forecast beats persistence on held-out data
A calibrated 80% interval with reported validation and test coverage	A guarantee that the interval contains the outcome
Local sensitivity and global permutation importance	A causal claim about the effect of weather or policy
Seasonal context: monthly medians, stagnation, and the hourly profile	A claim that winter afternoons are safe
A persistence comparison and a disagreement warning	A claim that the classifier is reliable for rare hazardous classes
A reproducible artifact pipeline and 14 tests	A live feed, a multi-day forecast, or a city-wide nowcast
Cautious, non-medical guidance	Medical advice or a substitute for official advisories

Table 34: Defensible and indefensible claims for the Deliverable 3 application.

11.5 Limitations and ethics

Table 35 consolidates the limitations from the model, the data, and the application. The list is long because the project is deliberately honest about what a single-station, six-input, one-day forecaster can do.

Ethics. The app processes no personal data and runs locally. If it were deployed publicly, it would need authentication where appropriate, rate limits, logging, a privacy review, and a clear separation between the demonstration and official health advice. The guidance is intentionally cautious: it tells users when to consider reducing exertion, not that a particular activity is safe. The single-station limitation is also an equity issue: a monitor at one location cannot represent the exposure of the whole city, and the dataset contains no demographic attributes with which to test fairness across groups.

11.6 Reproducibility and repository guide

Table 36 lists the commands needed to rebuild the application and the report from the committed source. All commands are run from the `deliverable_3` folder unless noted otherwise.

Limitation	Consequence	Mitigation or future work
One monitoring station	Results may not generalise to other neighbourhoods	Add stations and a spatial validation design
Regression ties persistence; classifier loses	No evidence of an accuracy gain	Report the baseline; treat the model as a demonstrator
Severe-tail under-prediction	The highest-risk days are the hardest	Cost-sensitive training, quantile models, conformal intervals
Six inputs and many defaults	The forecast is illustrative, not live	Ingest live weather and history; expose data provenance
Steady-state lags and rolling means	Rapid changes are under-represented	Use the actual previous days' $PM_{2.5}$
Classifier bias toward *Unhealthy*	Category probabilities are overconfident	Calibrate probabilities; use the implied category
80% interval covers 77.0% on test	Slight under-coverage	Recalibrate on more data; add conformal prediction
Rare hazardous classes	Per-class metrics are noisy	Collect more severe days; report support
Distribution shift	Validation-to-test reversals are possible	Rolling-origin validation; monitor drift
No traffic data	Calendar features are proxies	Integrate traffic and emissions data
Guidance is non-medical	Users may over-rely on the app	Keep the disclaimer; link to official advisories
Single-station environmental justice	No demographic or neighbourhood fairness claim	Add monitors and a population-aware evaluation

Table 35: Consolidated limitations and mitigations.

Step	Command	Expected output
Refit artifacts	`python -m src.train_pipeline`	Four joblib files and `metadata.json`
Run tests	`python -m unittest discover -s tests -t .`	14 tests pass
Run ablation	`python -m src.ablation`	`models/ablation.json`
Run app	`python -m streamlit run app/app.py`	Local browser UI
Build figures	`python report/make_figures.py`	Report figures and `examples.json`
Build report	`tectonic report_d3.tex`	`report_d3.pdf`

Table 36: Reproduction commands. The data files must be restored from the shared Drive or regenerated from the data scripts before the refit. Append `-v` to the test command for verbose output.

The artifact policy is the same as in Deliverable 2: no fitted model binary and no processed data file is committed. The `models/.gitignore` excludes the joblib files, `metadata.json`, and `ablation.json`; the repository root ignores the processed data. The refit is deterministic because the hyperparameters and the random seed are fixed. The report figures are generated from the persisted artifacts by `report/make_figures.py`, so the report and the app cannot silently diverge.

File map. The D3 source is small enough to audit: `app/app.py` is the interface; `src/inference.py` is the serving contract; `src/train_pipeline.py` refits the four pipelines and builds the metadata; `src/ablation.py` runs the nested feature-set comparison; `src/hourly.py` builds the hourly profile; `tests/test_inference.py` contains the 14 tests; `report/make_figures.py` builds the report figures; and `report/report_d3.tex` is this report. The D1 and D2 reports and code are unchanged.

11.7 Conclusion and future work

Deliverable 3 delivers a working, reproducible, explainable next-day air-quality application for Lahore. The user enters six values; the app returns a $PM_{2.5}$ forecast, two category views, an 80% interval, local and global explanations, seasonal context, and a persistence comparison. The model is the frozen Deliverable 2 model. It ties persistence on regression and loses to persistence on classification macro-F1. The report does not hide that result, and the app does not hide it either.

The most defensible contribution is the combination of a rigorous validation-only modelling protocol, a calibrated uncertainty layer, an explainable interface, a seasonal and hourly context, and a reproducibility policy that regenerates every artifact from committed scripts. The project is useful as a teaching and demonstration system, and as a foundation for a stronger operational model.

Future work should proceed in the order that the evidence demands. First, ingest live weather and $PM_{2.5}$ history so the inference layer does not rely on steady-state assumptions. Second, add a second station and a rolling-origin validation design to test whether the model generalises across place and time. Third, improve the severe tail with cost-sensitive training, quantile or conformal intervals, and a classifier that is calibrated rather than merely arg-maxed. Fourth, replace the one-at-a-time sensitivity with a Shapley-value decomposition if a more principled local explanation is needed. Finally, run a small user study before making any public-health claim. Until then, the app should remain a demonstration, and the official advisories should remain the source of actionable guidance.

A Viva questions and answers

What does the app predict?

It predicts tomorrow's $PM_{2.5}$ concentration and tomorrow's AQI category from six values for today: date, $PM_{2.5}$, AQI category, temperature, humidity, and wind speed. The forecast is always for the day after the selected feature date.

Why are there two AQI category views?

The classifier is a direct six-class model. The implied category converts the regression's predicted $PM_{2.5}$ through the same US-EPA breakpoints used to build the labels, so it is monotonic in the prediction. The classifier was weaker than persistence on test, so the app treats the implied category as the more defensible view and warns when the two disagree.

Does the model beat persistence?

No. On test, regression RMSE is 64.92 versus 64.82 for persistence, a 0.10 $\mu g/m^3$ difference and a skill score of -0.0016. Classification macro-F1 is 0.4780 versus 0.5157. Ridge reached a lower test RMSE (62.10), but it was not the validation-selected model, so switching to it after seeing test would violate the validation-only rule.

Why keep a model that does not beat persistence?

The project is an end-to-end explainable system, not an accuracy record. The model was selected on validation, the test result is reported honestly, and the application demonstrates inference, uncertainty, explanations, and reproducibility. Persistence is a strong one-day baseline because $PM_{2.5}$ is highly autocorrelated; failing to beat it is an informative result.

What is HGB?

Histogram-based gradient boosting. It builds an ensemble of small decision trees sequentially, with each tree correcting the errors of the previous ones. The histogram step bins continuous features for speed. It handles non-linear relationships, interactions, and missing values, and it needs little preprocessing. The frozen settings were selected on validation in Deliverable 2.

What is persistence?
Persistence assumes tomorrow will be the same as today: predicted value at $t+1$ equals the observed value at t. For classification, tomorrow's category equals today's. It is the simplest possible forecast and the baseline every model must beat.

Why macro-F1 instead of accuracy?
Accuracy is dominated by the common *Unhealthy* class. Macro-F1 gives each class equal weight, so it exposes poor performance on the rare hazardous classes. The HGB classifier has slightly higher test accuracy than persistence but lower macro-F1; reporting only accuracy would obscure the rare-class weakness.

What does the 80% interval mean?
It is a central interval for the next-day $PM_{2.5}$ observation, built from quantile HGB models at the 10th and 90th percentiles and calibrated on validation. Validation coverage is 80.3%; test coverage is 77.0%. The test split is used only to report coverage, never to choose the calibration factor.

Why is the interval so wide?
The model is uncertain, especially on severe days. The default November example forecasts 189.74 $\mu g/m^3$ with an interval of 133.8–294.1. The width is an honest statement about the evidence, not a bug. The backtest on 1 November 2024 shows that the actual value can fall outside the interval when the day is an extreme outlier.

Why only six inputs?
Usability. The remaining features use training medians or modes, and the $PM_{2.5}$ lags and rolling windows are held at today's value. The app calls itself an illustrative single-day forecaster for this reason. A production version would ingest live weather and a multi-day $PM_{2.5}$ history.

What changed after Deliverable 2?
The model and its reported scores are unchanged; the D3 refit reproduces D2 exactly. What is new is the application layer: inference, the Streamlit app, prediction intervals, explanations, the ablation, the hourly profile, the seasonal defaults, and the tests. The D2 code and report were not edited.

What is the biggest limitation?
The single monitoring station and the severe-tail under-prediction. The model is weakest on the highest-pollution days, which are exactly the days that matter most for public health. A second station, live inputs, and a cost-sensitive or conformal approach are the next steps.

B Glossary

Term	Meaning in this project
$PM_{2.5}$	Fine particulate matter with diameter at most 2.5 μm, measured in $\mu g/m^3$. The regression target.
AQI	Air Quality Index, a 0–500 scale computed from $PM_{2.5}$ using the US-EPA 2012 breakpoints.
HGB	Histogram-based gradient boosting; an ensemble of small trees trained sequentially to correct errors.
Regression	Predicting a continuous number, here tomorrow's $PM_{2.5}$.

Term	Meaning in this project
Classification	Predicting a discrete class, here tomorrow's AQI category.
Persistence	The baseline "tomorrow equals today."
RMSE	Root mean squared error; same units as the target and sensitive to large errors.
MAE	Mean absolute error; average absolute miss, in the target's units.
R^2	Fraction of variance explained; 1 is perfect and 0 is no better than the mean.
Macro-F1	The unweighted mean of per-class F1 scores; treats each class equally and exposes rare-class performance.
Weighted-F1	F1 weighted by class support; dominated by the common classes.
Precision and recall	Of the predicted positives, how many are correct; of the true positives, how many were found.
Confusion matrix	A table of predicted versus true classes.
Prediction interval	A range for a future observation, here the central 80%.
Quantile regression	Training a model to predict a conditional percentile rather than the mean.
Coverage	The fraction of true values that fall inside the interval.
Calibration	Adjusting interval width so empirical coverage matches the nominal level.
Permutation importance	Global importance: the increase in error when a feature is shuffled.
Sensitivity	Local importance: the change in the forecast when one input is perturbed.
Ablation	Removing feature groups to test whether they improve performance.
Data leakage	Information from validation or test contaminating training; prevented here by the seven-day purge and train-only preprocessing.
Class imbalance	Some categories are much rarer than others; handled with balanced class weights and macro-F1.
Piecewise constant	A tree model's prediction stays flat inside a leaf, so a small input change may have no effect.
Steady state	The app's assumption that today's $PM_{2.5}$ has been stable for the past week when it fills lags and rolling windows.

C File map and build notes

Table 38 lists the Deliverable 3 files a reviewer is most likely to open. The full repository map also includes the Deliverable 1 data scripts and the Deliverable 2 modelling code, which are unchanged.

Build notes. The report is compiled from the `deliverable_3/report` directory with Tectonic:

```
tectonic report_d3.tex
```

File	Role
app/app.py	Streamlit interface: six inputs, forecast cards, interval, guidance, charts, expanders
src/inference.py	Loads the four pipelines and metadata; assembles the feature row; predicts $PM_{2.5}$, category, interval, explanation, and guidance
src/train_pipeline.py	Refits the frozen pipelines; writes the joblib artifacts and metadata.json
src/ablation.py	Runs the nested history/calendar/weather ablation
src/hourly.py	Builds the hour-of-day and seasonal $PM_{2.5}$ profile
tests/test_inference.py	Fourteen unit tests for the inference and app contract
report/make_figures.py	Generates the report figures and the worked examples from the frozen artifacts
report/report_d3.tex	This report's master LaTeX file
report/phase34_final.tex	Deliverable 2 Phase 3–4 body, reused with adjusted table paths
report/phase56_final.tex	New Deliverable 3 Phase 5–6 chapters
report/appendix_final.tex	Viva questions, glossary, and file map
README.md	Install, run, artifact, and submission notes
STUDY_REPORT.md	Plain-language technical study report
requirements.txt	Python dependencies for the application layer

Table 38: Deliverable 3 file map.

The report/make_figures.py script must be run first if the figures or examples.json are missing. The model artifacts must be present for the script to compute the worked examples; run python -m src.train_pipeline from deliverable_3 first. The report's numerical tables are taken from the frozen D2 metrics and the D3 metadata, ablation, and worked-example files.

References

[1] R. Majeed, M. S. Anjum, M. Imad-ud-din, S. Malik, M. N. Anwar, B. Anwar, and M. F. Khokhar, "Solving the mysteries of Lahore smog: the fifth season in the country," *Frontiers in Sustainable Cities*, vol. 5, art. 1314426, 2024. doi:10.3389/frsc.2023.1314426.

[2] World Health Organization, "Ambient (outdoor) air pollution," fact sheet, 24 October 2024. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/ambient-(outdoor)-air-quality-and-health

[3] U.S. Environmental Protection Agency, "Particulate Matter (PM) Basics." [Online]. Available: https://www.epa.gov/pm-pollution/particulate-matter-pm-basics

[4] U.S. Environmental Protection Agency, "Technical Assistance Document for the Reporting of Daily Air Quality – the Air Quality Index (AQI)," EPA-454/B-18-007, September 2018. [Online]. Available: https://www.airnow.gov/sites/default/files/2020-05/aqi-technical-assistance-document-sept2018.pdf

[5] H. M. S. Waqas, T. Naz, K. Shahid, A. Yaqoob, and M. B. Ahmad, "Time series forecasting of air quality index in Lahore: a machine learning perspective with Facebook Prophet model," *International Journal of Scientific Research in Science and Technology*, vol. 11, no. 5, pp. 219–226, 2024. doi:10.32628/IJSRST2411598.

[6] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[7] J. H. Friedman, "Greedy function approximation: a gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[8] Streamlit, "Streamlit documentation," 2026. [Online]. Available: https://docs.streamlit.io/